SwiftUI 界面设计自学手册

设计规范 · 布局武器库 · 十个界面任务卡

用于独立完成《物品管理 App》的界面设计与实现

配套文档：《物品管理 App 创意方案集》

适配环境：Xcode 27.0 · Swift 6.4 · iOS 27 SDK · SwiftUI

目 录

第 0 章 这份手册怎么用 5

0.1 它解决什么问题 5

0.2 哪些是我给你的，哪些要你自己设计 5

0.3 学习路径与时间安排 6

0.4 阅读约定 6

第 1 章 界面设计的最小知识集 8

1.1 好界面的四条底线 8

1.2 设计的语法：间距、字号、圆角、颜色 9

1.3 一套可以直接用的设计规范 10

1.4 深色模式与可访问性 12

1.5 新手最常见的七个审美错误 13

第 2 章 从需求到界面：可复用的设计流程 16

2.1 五步法 16

2.2 信息优先级：一个界面只讲一件事 17

2.3 导航结构怎么选 18

2.4 画线框：用纸和 ASCII 就够 19

2.5 从线框到 SwiftUI 代码的翻译对照表 21

第 3 章 SwiftUI 布局与样式的完整武器库 23

3.1 布局的两个核心概念 23

3.2 容器与排列 25

3.3 修饰符的顺序为什么重要 29

3.4 间距、内边距与留白 31

3.5 背景、圆角、边框与阴影 33

3.6 文字排版 36

3.7 颜色系统 38

3.8 图片与 SF Symbols 40

3.9 常用控件速查 42

3.10 反馈与状态提示.....46
3.11 动效入门.....48
3.12 用 #Preview 高效迭代设计.....50
第 4 章 物品管理 App 的界面地图.....54
4.1 界面地图与难度分级.....54
4.2 数据模型与界面字段对照.....55
4.3 导航结构设计.....56
4.4 先做哪个界面：推荐顺序.....57
第 5 章 十个界面设计任务卡.....59
5.1 怎么用任务卡.....59
5.2 任务卡一：物品列表页.....60
5.3 任务卡二：物品详情页.....62
5.4 任务卡三：新增与编辑表单.....64
5.5 任务卡四：分类管理.....66
5.6 任务卡五：搜索与筛选.....68
5.7 任务卡六：空状态与首次引导.....70
5.8 任务卡七：统计页.....71
5.9 任务卡八：设置页.....73
5.10 任务卡九：删除与撤销.....75
5.11 任务卡十：分享卡片页.....76
第 6 章 自己造可复用的组件.....79
6.1 什么时候该把界面拆成组件.....79
6.2 从硬编码到可配置：一个物品卡片的演进.....79
6.3 自定义 ViewModifier 与 ButtonStyle.....83
6.4 组件 API 的设计原则.....84
第 7 章 设计自查清单.....86
7.1 布局与间距.....86
7.2 内容与状态完整性.....86
7.3 视觉与配色.....87
7.4 交互与反馈.....87

7.5 适配与可访问性.....88
第 8 章 独立设计挑战.....89
8.1 挑战规则.....89
8.2 挑战一：把列表改成网格.....89
8.3 挑战二：设计临期看板.....90
8.4 挑战三：设计筛选面板.....90
8.5 挑战四：设计统计卡片.....91
8.6 挑战五：设计分享卡片.....91
8.7 挑战六：设计首次使用引导.....91
8.8 挑战七：给同一个界面做两套风格.....92
8.9 挑战的验收标准.....92
第 9 章 设计工作表与资源.....94
9.1 界面设计工作表模板.....94
9.2 设计规范速查卡.....94
9.3 组件登记表.....96
9.4 常用资源与工具.....96

第 0 章 这份手册怎么用

0.1 它解决什么问题

你已经会写 Swift，也能照着别人的代码抄出一个能跑的界面。但把"照着抄"换成"自己设计"，中间卡着几件没人教过的事：

- 一个界面上到底该放哪些信息，哪些可以砍掉。
- 间距取 16 还是 12，字号用 .headline 还是 .title3，全凭感觉。
- 布局写出来总是差一点，说不清哪里别扭，也不知道怎么改。
- 知道列表、表单、卡片大概怎么写，但不知道一个真实 App 的界面该怎么组织起来。

这份手册把这几件事拆开教：第 1 章给你一套确定的数值规范，让你不用再靠感觉；第 2 章给你一套可复用的设计流程；第 3 章是可以随时翻的布局查字典；第 5 章的十张任务卡直接对应你要做的十个界面。第 8 章则是留给你的部分——只给需求，不给代码。

0.2 哪些是我给你的，哪些要你自己设计

这是这份手册最重要的约定，请先看清楚，后面每一章都按这个分工来写。

我给你的部分：

我给的东西	在第几章
一套可以直接用的数值规范（间距、字号、圆角、颜色）	第 1 章 1.3 节
判断界面好坏的标准和常见错误	第 1 章 1.1、1.5 节
从需求到界面的五步流程和线框画法	第 2 章
SwiftUI 布局与样式的能力清单和用法	第 3 章
界面的完整清单、导航结构和开发顺序	第 4 章
每个界面的需求、必须包含的信息、验收标准	第 5 章
把重复界面抽成组件的方法	第 6 章
自查清单和独立设计挑战	第 7 章、第 8 章

你要自己做的部分：

你决定的事	说明
每个界面的具体布局	任务卡只给"必须出现什么"和推荐结构，具体怎么排由你定

你决定的事	说明
视觉风格	用哪张图当详情页头图、状态色怎么用、信息是分成卡片还是排成列表
信息取舍	一行里放三项还是五项，次要信息是折叠还是省略
组件怎么拆	哪些界面元素值得抽成组件、参数怎么设计
空状态、错误态、加载态的表现	任务卡只要求"必须处理"，怎么呈现是你的设计
动效与细节	加不加动画、加在哪里、点击反馈怎么做

换句话说：**我给约束和依据，你给判断和表达。** 任务卡里的"起步代码"永远只是脚手架，里面一定有标着"留给你设计"的注释，那些地方默认是空的，等你填。

> 不要跳过自己的判断直接抄起步代码。抄出来的界面能跑，但十个界面会长得一模一样，而且你学不到任何东西。这份手册的价值在于让你做出自己的决定，并且有依据。

0.3 学习路径与时间安排

按你手上的时间选一条。

只有两天，要马上开始写界面：读第 1 章的 1.3 节（规范）和第 1 章的 1.5 节（常见错误），然后直接进第 5 章，从任务卡一往下做，卡在语法上就翻第 3 章对应的那一节。

有一周，想认真设计：第 1 章通读（半天），第 2 章通读并动手画三个线框（半天），第 3 章按需查阅（边做边看），第 4 章通读了解全貌（一小时），然后按第 5 章的任务卡逐个完成（三到四天），最后用第 7 章自查、用第 8 章做两个挑战（一天）。

想做出能拿得出手的作品：在上一档的基础上，加读第 6 章并把界面重构成组件（半天），完成第 8 章的全部七个挑战（一到两天）。

无论走哪条路，顺序原则是一样的：**先读规范，再画线框，最后写代码。** 直接打开 Xcode 边想边写，是新手界面看起来乱的主要原因。

0.4 阅读约定

- 正文中的等宽字体表示代码、标识符或文件名，例如 `ItemCard`、`@Query`。
- 灰色底纹的代码块都是可直接运行的 SwiftUI 代码，但第 5 章任务卡里的代码是**骨架**，留白处需要你自己补全。

- 所有数值（间距、字号、圆角、颜色）都来自第 1 章 1.3 节的设计规范，正文里不再重复解释出处。
- 数据模型的字段名统一使用 `name`、`categoryName`、`quantity`、`expireDate`、`location`、`note`、`isFavorite`、`tags`、`createdAt`、`imageData`，完整定义见第 4 章 4.2 节。
- 技术基线为 iOS 17 及以上、SwiftUI、SwiftData，环境为 Xcode 27.0、Swift 6.4、iOS 27 SDK。手册不会教 UIKit 和 Storyboard。
- 配套文档《物品管理 App 创意方案集》负责选题、功能拆解和实施排期，这份手册只管界面。

> 一个建议：每做完一个界面，回到第 7 章的自查清单过一遍，再去截图。养成"做完就自查"的习惯，比事后统一返工省时间得多。

第 1 章 界面设计的最小知识集

界面设计不是天赋。人眼对"整齐、克制、有主次"的画面有稳定的偏好，这些偏好可以被翻译成几条硬规则。这一章把这些规则变成你写代码时能随时查的数值，看完你应该能对着自己的界面说出"哪里不对、为什么不对、改成多少"，而不是"多看看别人的 App 就有感觉了"。

1.1 好界面的四条底线

下面四条是底线，不是加分项。任何一条破了，界面就进入"业余"区间，哪怕功能完全正确。

一、对齐。 人眼把左边缘对齐的一列内容自动读成一个整体，把没有对齐的内容读成散落的碎块。列表页里所有文字左边缘在同一条竖直线上，视线就能沿直线往下扫；只要有一个元素偏了 6 点，视线就要左右折一下，整页看起来"歪了"，但用户说不出哪里歪。所以规则是：一屏之内左边缘越少越好，通常只留两条——页面边距一条，卡片内部内容一条。

```
// 反例：卡片左右边距 12，卡片内部又没有统一内边距，两列文字左边缘对不上
VStack(alignment: .leading, spacing: 8) {
    Image(systemName: "shippingbox")
    Text(item.name).font(.headline)
}

// 正例：内边距用规范值，图标与文字之间的间距也固定，缩略图列宽统一
HStack(spacing: 8) {
    ThumbnailView(data: item.imageData)
        .frame(width: 56, height: 56)
    VStack(alignment: .leading, spacing: 4) {
        Text(item.name).font(.headline)
        Text(item.categoryName).font(.subheadline).foregroundStyle(.secondary)
    }
}
.padding(12)
```

二、间距成体系。 两个元素之间的距离在告诉用户"它们有多亲"：距离小表示同属一组，距离大表示分属两段。如果全屏间距都是 10 点，用户就看不出哪几个字段属于同一个物品，只能靠读文字去分辨，认知负担直接翻倍。反例是列表行上下 10、行内图标与文字之间也是 10，行内和行间一样宽，整页看起来像一整块文字；正例是行内用 8、行与行之间用 10、卡片与卡片之间用 12，三个数值各说一件事。所以间距必须从固定刻度里取，让 8 / 12 / 16 / 24 四档各自表达一种关系强度。

三、层级分明。 人眼进入一个新界面时，会按"最大、最黑、对比最强"的顺序扫描，先抓到一个东西，再抓下一个。如果一屏里十行文字全是 17 号常规字重，用户的视线没有落点，只能从头读到尾才能知道这页在讲什么。反过来，把物品名称设成 `.headline`、分类和日期设成 `.subheadline` 加

`.secondary`，用户 0.5 秒内就能确认"这是我要找的那件东西"。层级靠字重、字号、颜色一起拉，不要只调大小。

```
// 层级靠字重、字号、颜色三个维度一起拉
VStack(alignment: .leading, spacing: 4) {
    Text(item.name).font(.headline).foregroundStyle(.primary)
    Text(item.categoryName).font(.subheadline).foregroundStyle(.secondary)
    Text(item.updatedAt, format: .dateTime.month().day())
        .font(.caption)
        .foregroundStyle(.tertiary)
}
```

四、全 App 一致。 用户在第三个页面之后就不再读界面了，而是靠肌肉记忆操作：返回键在左上角、新建按钮在右上角、保存按钮在表单最下方。同一个动作在不同页面换了位置或换了样子，用户每次都要重新搜一遍，操作会明显变慢，还容易误点。反例是列表页用右上角的加号新建、统计页却在页面中间放一个"新建"按钮；正例是把同一件事收敛成一个可复用的工具栏组件，哪个页面要就挂哪个页面：

```
// 新增入口统一成一个修饰符，各页面挂同一份，位置和图标永远不会走偏
extension View {
    func addItemToolbar(action: @escaping () -> Void) -> some View {
        toolbar {
            ToolbarItem(placement: .primaryAction) {
                Button("添加", systemImage: "plus", action: action)
            }
        }
    }
}
```

一致性还包含视觉层面：同一个"卡片"概念，圆角和底色在全 App 必须是同一个值。

下面这条判断标准可以在写代码时随时用：如果你需要在两个页面里解释"为什么这里和那里不一样"，那多半就不该不一样。

1.2 设计的语法：间距、字号、圆角、颜色

上一节的四条底线要靠具体数值落地，而数值最容易犯的错是"每个界面重新拍脑袋"。这一节解释为什么要用刻度。

原因很直接：设计是在有限选项里做取舍，选项太多反而选不出来。如果你每次都能写任意数值，你在每个元素上都要重新判断一次"这里 13 还是 14"，这些判断互相冲突，最后整页没有秩序。改成从刻度里取，你只判断"这两个元素的关系属于哪一档"，判断量少一个数量级，结果反而更整齐。

间距用 4 的倍数。 4 点是 iOS 上能被视觉稳定感知的最小差距，2 点的差异人眼几乎分不出来，但会让代码里出现一堆 14、18、22 这种脏数值。具体取值见 1.3 的规范表。

> 规则：写不出这个间距属于哪一档，就说明这个间距不该存在。宁可删掉一个 padding，也不要凭空造一个 13。

字号用系统阶梯。 不要出现 `.system(size: 15)`、`.system(size: 19)` 这种写法。系统语义字号本身就是一套阶梯：`.largeTitle`、`.title`、`.title2`、`.title3`、`.headline`、`.body`、`.callout`、`.subheadline`、`.footnote`、`.caption`、`.caption2`。相邻两档之间的差距是系统调过的，你随便插一个中间值，只会让层级变模糊。同时语义字号会跟着系统的"文字大小"设置自动缩放，写死磅值就丢掉这个能力。

圆角只留两种。 圆角是"这个元素是什么"的身份标记：小圆角意味着可以点的小控件或缩略图，大圆角意味着成块的容器。一个界面出现四五种圆角，用户读不出这些身份，卡片看起来就像随便糊上去的。规范里只留 10 和 16 两种，需要更强的按钮时用胶囊形。

颜色只分角色。 颜色不是装饰，是信息。你只需要四类：文字层级色（`.primary`、`.secondary`、`.tertiary`）、页面与容器底色、一个全局强调色、表达状态的状态色。多出来的颜色会让用户试图从颜色里读出含义，而那个含义并不存在。

一句话判断标准：**如果你正在输入的数值，无法在规范表里指出它对应哪一行，就停下来换成表里最接近的那一档。**

1.3 一套可以直接用的设计规范

一句话结论：下面这套数值就是本 App 的设计规范，所有界面都从这里取值，不要临场拍脑袋。

间距，只用 4 的倍数。

位置	取值
图标与文字之间	8
卡片内部留白	12 或 16
列表行上下留白	10
页面左右边距	16

位置	取值
卡片与卡片之间	12
区块与区块之间	24

圆角，一个界面最多两种。

元素	取值
列表缩略图	10
卡片、大图	16
按钮、输入框	10，或者整条胶囊形

字号，用系统的语义字号，不写死磅值。

用途	SwiftUI 写法
页面主标题	.largeTitle
区块标题	.title3 或 .headline
正文与列表主文本	.body
次要信息	.subheadline 或 .footnote
时间、标签、角标	.caption 或 .caption2

规则：一个界面最多出现 4 种字号，标题和正文之间至少差一个档位。用语义字号的好处是它会跟着系统的"文字大小"设置自动缩放，写死 .system(size: 14) 就失去了这个能力。

颜色，只用语义色和角色色。

角色	用法
主文字	.primary
次要文字	.secondary
更弱的说明	.tertiary
页面底色	Color(.systemGroupedBackground)
卡片底色	Color(.secondarySystemGroupedBackground)
强调色	在根视图用一句 .tint(.blue) 统一设置
状态色	正常 .green、临期 .orange、过期 .red、已归档 .secondary

规则：一个界面最多 1 个强调色加 3 个状态色，状态色只用来表达状态，不要拿它当装饰。永远不要写 Color.white 当背景、Color.black 当文字，那样在深色模式下会直接失效。

图标与图片。

场景	规格
列表缩略图	56 x 56，圆角 10，裁剪填充
网格卡片封面	正方形，圆角 16
详情页头图	宽度撑满，高度 220
空状态插画图标	系统符号，字号 .largeTitle 的 2 到 3 倍

两条容易被忽略的硬规则。

- 可点区域最小 44 x 44。图标本身可以只有 20 点，但要给它加 .frame(minWidth: 44, minHeight: 44) 或者 .contentShape。
- 阴影尽量不用。层级关系优先靠"分组背景色差、圆角、间距"来表达；确实需要浮起感时，用 .shadow(color: .black.opacity(0.06), radius: 8, y: 2) 这种极轻的阴影，不要用默认的大黑影。

1.4 深色模式与可访问性

这两件事不是"做完界面再补的优化"，它们是写第一行样式代码时就必须做对的选择。做错的话，后面每加一个界面都要返工。

一、颜色必须用语义色，深色模式才会自动适配。 你写 `Color.white` 当卡片底色、`Color.black` 当正文颜色，浅色模式下完全正常，用户一切到深色模式，白底黑字和系统背景撞在一起，界面变成一块发光的白板。语义色的含义是"跟随系统主题变化的那个角色"，它自己有浅色和深色两套值。

```
// 反例：写死具体颜色
Text(item.name).foregroundStyle(.black)
Color.white.padding(12).clipShape(RoundedRectangle(cornerRadius: 16))

// 正例：用角色色，两套主题自动生效
Text(item.name).foregroundStyle(.primary)
Color(.secondarySystemGroupedBackground)
    .padding(12)
    .clipShape(RoundedRectangle(cornerRadius: 16))
```

唯一可以写具体颜色值的地方是状态色（`.green`、`.orange`、`.red`），因为它们本来就表达了明确语义。即便如此，也要保证它们在深色下仍然能看清。测试方法很简单：把 App 在"外观 - 深色"下跑一遍，凡是出现看不清的文字、糊成一片的卡片、和背景同色的分隔线，都是写死了颜色。

二、字号要用语义字号，让动态字体生效。 用户可以在系统设置里把文字调大，视力不好的用户经常拉到最大。动态字体（Dynamic Type）就会按你写的语义字号自动放大。`Text` 默认就是 `.body`，

已经跟随系统；你要避免的是手写 `.font(.system(size: 14))`。放大之后布局会不会挤坏，也要实测：把文字大小调到最大，如果卡片里的名称被截断成"相机电…"，就给它加上行数上限和缩放策略。

```
Text(item.name)
    .font(.headline)
    .lineLimit(2)                       // 先限制行数，避免整行被撑高或截断
    .minimumScaleFactor(0.8)            // 空间实在不够时允许缩到 80%
```

三、对比度决定"能不能读"。 正文文字和它背后底色之间的对比度至少要达到 4.5:1，大号标题可以放宽到 3:1。具体操作上你不需要背公式：正文一律用 `.primary`，次要信息一律用 `.secondary`；不要在浅灰底色上再叠 `.secondary` 文字；`.tertiary` 只配给角标、时间戳这类辅助信息。

四、可点区域不小于 44 x 44 点。 这是给手指留的余量，不是给眼睛看的尺寸。一个 20 点的删除图标，视觉上没问题，但用户点的时候是靠记忆去戳那个位置，10 次里总有一两次点空。修法是给可点元素加最小尺寸，并让它整块响应点击。

```
Button {
    favorite(item)
} label: {
    Image(systemName: item.isFavorite ? "star.fill" : "star")
        .frame(minWidth: 44, minHeight: 44)   // 撑到最小可点尺寸，图标仍保持 20 点
        .contentShape(Rectangle())            // 让整块区域都能接收点击，而不只是图形本身
}
```

最后是 VoiceOver：给纯图标按钮加 `.accessibilityLabel("收藏")`，给图片加一句描述，用 `.accessibilityElement(children: .combine)` 让一张卡片被读成一段话而不是五段碎片。这一段先记住"图标必须有无障碍标签"。

1.5 新手最常见的七个审美错误

七个错误都不涉及审美水平，全是可修的技术问题。

错误一：一个界面用了六种字号。 错误的样子 —— 标题 22、副标题 18、正文 15、说明 13、日期 12、角标 11，全部手写磅值。结果整页没有主次。应该怎么做 —— 压到三种以内，只用语义字号：页面标题用 `.largeTitle`，区块标题用 `.title3` 或 `.headline`，正文用 `.body`，次要信息用 `.subheadline` 或 `.caption`。

错误二：颜色太多。 错误的样子 —— 蓝色标题、绿色图标、橙色数字、红色标签、紫色按钮。应该怎么做 —— 全 App 一个强调色，在根部用 `.tint(.blue)` 统一；文字层级用 `.primary`、`.secondary`、`.tertiary`；只有表达状态时才出现 `.orange` 和 `.red`。

错误三：间距不成倍数。 错误的样子 —— 一屏里出现 5、7、9、13、15，多数是"看着差不多"随手写的，差值人眼分辨不出，却让列表看起来脏、卡片看起来歪。应该怎么做 —— 全部改成 4 的倍数，同一层次保持一致：列表行上下留白 10，卡片内边距 12，页面左右边距统一 16。

错误四：什么都加边框和阴影。 错误的样子 —— 每个元素都描边、都投影，页面里全是线和灰块，像没做完的模型图。应该怎么做 —— 用"分组背景色差加圆角加间距"表达层级：卡片用 `Color(.secondarySystemGroupedBackground)` 加圆角 16，放在 `Color(.systemGroupedBackground)` 页面上，两者都不描边也不投影。

错误五：标题重复啰嗦。 错误的样子 —— 大标题写"物品列表"，下面又写"以下是你的所有物品"，再下面再写"物品"。同一件事说了三遍。应该怎么做 —— 页面只需要一个标题，用 `navigationTitle` 或 `.largeTitle`，其余全部删掉；需要分组时用 `Section` 的自动分组标题。顺便提醒：`.grouped` 样式的 `Form` 里分组标题会自动变成大写，想保持原样加一句 `.textCase(nil)`。

错误六：按钮和点击区域太小。 错误的样子 —— 一个 20 点的删除图标直接当按钮，没有外边距，两个图标还挨在一起，用户点删除总是点到旁边的收藏。应该怎么做 —— 可点区域最小 44 x 44，用 `.frame(minWidth: 44, minHeight: 44)` 撑开，再用 `.contentShape(Rectangle())` 让整块可点。

```
// 反例：只有图标本身能点，且两个按钮间距为 0
HStack(spacing: 0) {
    Button { favorite(item) } label: { Image(systemName: "star") }
    Button { delete(item) } label: { Image(systemName: "trash") }
}

// 正例：各自撑满 44 x 44，整块可点，间距 8
HStack(spacing: 8) {
    Button { favorite(item) } label: {
        Image(systemName: "star")
            .frame(minWidth: 44, minHeight: 44)
            .contentShape(Rectangle())
    }
    Button { delete(item) } label: {
        Image(systemName: "trash")
            .frame(minWidth: 44, minHeight: 44)
            .contentShape(Rectangle())
```

```
    }
}
```

错误七：深色模式下颜色失效。 错误的样子 —— 卡片底色写 `Color.white`、正文写 `Color.black`，浅色模式好看，深色模式下一片惨白，或者浅灰底上的浅灰字完全读不出来。应该怎么做 —— 底色用 `Color(.systemGroupedBackground)` 和 `Color(.secondarySystemGroupedBackground)`，文字用 `.primary`、`.secondary`、`.tertiary`，分隔线交给 `List` 自动处理。每做完一个界面都切一次深色模式跑一遍。

第 2 章 从需求到界面：可复用的设计流程

第 1 章给了你一整套数值，但数值本身不告诉你"这个界面上该放什么"。从"我要做一个物品管理 App"到能写代码，中间缺的是一套固定动作。这一章给出一套五步流程，每一步都有明确产出物，做完之后写代码只是把纸上的东西翻译成 SwiftUI。

2.1 五步法

这五步的顺序不能换，也不能合并。核心原则是：**先用最便宜的方式试错，再写代码。** 纸上的修改成本是零，代码里的修改成本最高。

第一步：列信息。 动作是把这件事涉及的所有信息项全部写下来，只求全，不求对。产出物是一张信息清单。物品列表页大致会写出：名称、图片、分类、数量、存放位置、到期日期、是否收藏、备注、标签、创建时间、更新时间、临期状态。时间：10 分钟。

第二步：排优先级。 动作是把清单里的每一项标成"必须有"和"可以有"，再按"用户来这个页面最想干的一件事"把"必须有"的几项排个序。产出物是一张带顺序的表。时间：15 分钟。

第三步：画线框。 动作是用纸笔或纯文本把页面的区块位置画出来，标上优先级顺序、可点区域、状态变化。产出物是列表页、详情页、表单页各一张草图。时间：每张 20 分钟，详细做法见 2.4。

第四步：定组件。 动作是给线框里每个块指定对应的 SwiftUI 组件：这一块是 `List` 的一行，那一块是 `Section`，右上角那个是 `toolbar` 里的按钮，底部是 `TabView`。产出物就是 2.5 节那张对照表。时间：15 分钟。

第五步：写代码。 动作是照着线框和对照表写 SwiftUI，结构一次成型，只剩下调间距和圆角。这一步才允许你打开 Xcode。

步骤	具体动作	产出物	大致耗时
列信息	把涉及的信息项全部写下来，只求全	一张信息清单	10 分钟
排优先级	标必须有和可以有，再按用户目的排序	带顺序的优先级表	15 分钟
画线框	画区块位置，标顺序、可点区、状态变化	每页一张线框草图	每页 20 分钟
定组件	给每个块指定对应的 SwiftUI 组件	块到组件的对照表	15 分钟
写代码	照线框和对照表落地	可运行的界面	按页面复杂度

为什么不能跳过线框直接写代码。 有三个具体原因。第一，写代码时你的注意力被语法和预览报错吃掉，没有余量去想结构，于是你只会把"手边最方便写的组件"塞进去，而不是把"最该突出的信息"放上去。第二，改结构在代码里比在纸上贵得多：纸上挪一个字段是划掉重画，代码里要拆掉 `HStack`、重算间距、再看预览，五分钟起步，于是你会不自觉地说"算了就这样吧"。第三，线框会提前暴露信息量问题——画的时候你才会发现"这一行塞了六个字段，根本放不下"，这在纸上是 30 秒的事，在代码里意味着重新设计卡片高度、图片尺寸和整个列表行结构。

> 结论：没有线框就开始写 SwiftUI，等于把结构决策推迟到最贵的时候做。

2.2 信息优先级：一个界面只讲一件事

排优先级的唯一依据是一句话：**用户打开这个界面，最想干成的那一件事是什么。** 物品列表页的答案是"找到那件东西"，不是"欣赏你的收藏总量"，也不是"了解所有物品的完整信息"。服务于这一件事的信息留下，其余砍掉。

先看物品列表页。一行里必须留的只有三项：图片、名称、一个状态信号（比如临期或过期）。用户靠在列表里快速扫名称来定位物品，图片帮他确认"是这个东西"，状态信号决定他要不要立刻处理。可以留但必须变弱的：分类、数量。分类已经在筛选器里出现过，行内再强调是重复。可以砍的：备注、标签、创建时间、更新时间。备注往往很长，一行放不下；标签数量不定，会让行高忽高忽低，列表一乱就整个扫不动，想留就留到详情页。

优先级排序的机械做法：给每一项在"对用户目的的重要度（1 到 5）"和"展示成本（占用的高度或宽度，1 到 5）"上打分，重要度高而成本低的排前面。

信息	重要度	展示成本	结论
名称	5	2	第一优先级，用 .headline，最多两行
图片	4	3	第一优先级，56 x 56 缩略图
到期状态	5	1	第一优先级，用状态色小字或角标
分类	3	1	第二优先级，.subheadline 加次要色
数量	2	1	第二优先级，只在大于 1 时显示
存放位置	2	1	第三优先级，移入详情页
备注	3	5	第三优先级，移入详情页
标签	2	4	第三优先级，移入详情页
创建时间	1	1	直接砍掉，改用排序而不是展示

同样的方法用在详情页。详情页最想干的一件事是"确认这件东西的完整情况并决定下一步动作"，所以名称和图片放大到页首，状态和到期日期紧跟其后，位置和备注放进下方分组，编辑、删除、收藏统一收进工具栏。

> 判断标准：如果一行里的信息让你无法在 2 秒内说出"这一行最重要的是什么"，就说明这一行还没有排优先级。

2.3 导航结构怎么选

导航方式决定了用户的心理地图。选错的后果不是样式不统一，而是用户不知道自己在这一层还是上一层。

方式	什么时候用	什么时候别用	用户预期
TabView	两到五个互不依赖的顶级板块	把同一层级的第五个以上功能硬塞进去	切换页签不丢上下文，回到页签时还在原来位置
NavigationStack 层级	从列表进入详情，可以继续往下钻	平级的、没有从属关系的页面	从右滑入，左上角能退回上一层
sheet	短任务：新增、编辑、选择器	需要再往下钻两三层的大流程	从下浮出，能下拉或点取消放弃，不离开当前上下文
fullScreenCover	沉浸式、必须完整走完的流程	日常的增删改查	全屏盖住一切，必须有明确的关闭或完成出口

本 App 的推荐结构：根视图是 `TabView`，三个页签分别是物品、统计、设置；每个页签各自包一层 `NavigationStack`，页签内部的浏览历史互不干扰；物品页内部用层级导航，从列表页进入详情页；新增和编辑用 `sheet`，删除确认用 `alert`。

```
struct RootTabView: View {
    var body: some View {
        TabView {
            Tab("物品", systemImage: "shippingbox") {
                ItemListView()
            }
            Tab("统计", systemImage: "chart.bar") {
                StatsView()
            }
            Tab("设置", systemImage: "gearshape") {
                SettingsView()
            }
        }
    }
}
```

为什么不用四个页签。 每个页签都在向用户承诺"这里有一整块独立的功能"。把"分类管理"做成第四个页签，用户会期待它像物品列表一样有完整的浏览、搜索和批量操作，点进去却发现"就这"；四个页签的标题也开始变挤，中文标题在窄屏上容易被截断。分类管理放进设置页作为一行可进入的配置项，符合用户"这属于偏好设置"的预期。

为什么编辑用 sheet 而不是新页面。 编辑是有明确终点的短任务：进来、改几个字段、保存或取消、回到原处。sheet 天生表达这个形状——从下浮出说明"这是一层临时操作"，下拉就能放弃说明"不保存也不会迷路"。如果编辑做成 push 出来的新页面：一是用户会以为进入了物品的另一个展示层，于是期待这里也有完整的物品信息，而表单里只有输入框；二是"取消"在一层层进入的页面里没有自然位置，右滑返回是"后退"，而用户想表达的是"我放弃这次修改"。

2.4 画线框：用纸和 ASCII 就够

线框（Wireframe）是页面的骨架图，只画方块和文字，不画颜色和真实图片。工具不重要：格子纸、白板、纯文本都行，纯文本的好处是可以直接贴在代码旁边，改起来几秒钟。

画的时候必须标出四样东西，缺一样这张线框就还没画完：

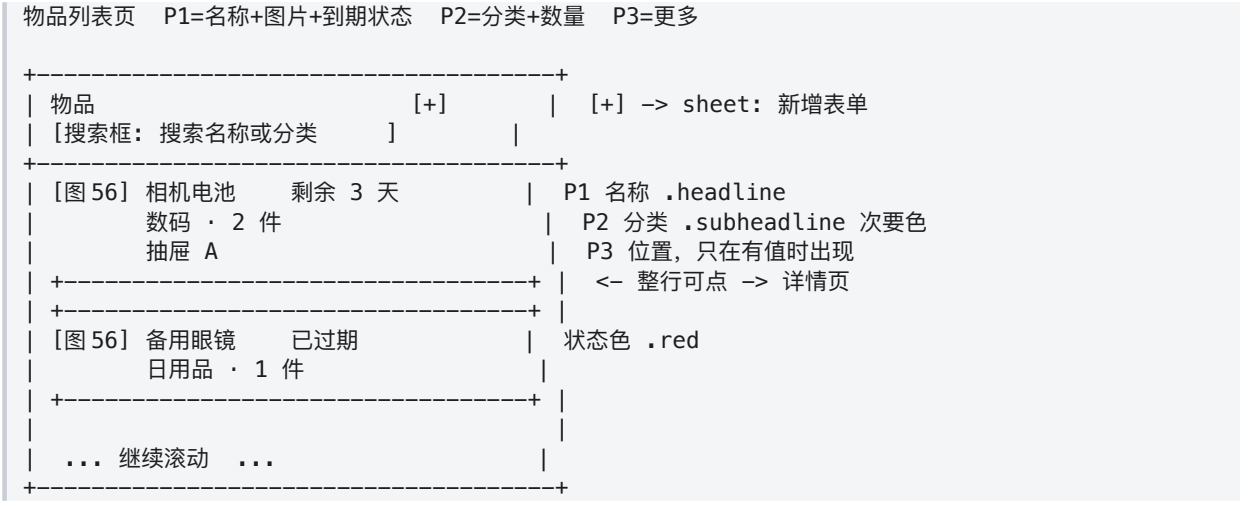
1. 信息块：每一块是什么内容，写具体中文，不要写"内容区"这种废话。

2. 优先级顺序：用 P1、P2、P3 标出来，和 2.2 那张表对得上。

3. 可点区域：用方括号框出来，标上点了会去哪。

4. 状态变化：空列表、图片加载失败、过期物品分别长什么样。

下面是三个画完的线框：物品列表页、新增表单页和空状态。



```
|  物品      |     统计      |     设置     |  底部 TabView，切页签不丢状态
+--------------------------------------+
```

这个列表页这样排有三个理由。第一，加号放在标题右侧而不是底部，因为新增是随时可能发生的动作，固定在工具栏里比藏在列表末尾好找。第二，搜索框占一整行，因为它服务的正是"找到那件东西"这个主目的。第三，日期和状态合并成右侧一列短文本，不分两行，否则每行高出一档，一屏少显示两条。

```
新增表单页（sheet）   P1=名称   P2=图片+分类   P3=其余字段

+--------------------------------------+
|  取消          新增物品          保存     |  取消=关闭不保存   保存=校验后写入
+--------------------------------------+
| [ 点击添加图片 ]                        |  P2 放在名称之后，因为先有名字再配图
|                                        |
| 名称                                   |
| [ 例如：相机电池                    ]   |  P1 唯一必填项
|                                        |
| 分类                                   |
| [ 数码                           v ]    |  P2 选择器，默认"未分类"
|                                        |
| 数量                                   |
| [ − ]         1         [ + ]          |  P2 步进器，最小 1
|                                        |
| 到期日期                        [开关]   |  P3 开关关闭时隐藏下面的日期行
| [ 2027-03-01                        ]  |      这是"状态变化"的一种
|                                        |
| 存放位置                                 |
| [                                   ]  |  P3
|                                        |
| 备注                                   |
| [                                   ]  |  P3 多行，放最后
+--------------------------------------+
```

表单页的关键在于顺序和状态。顺序按"填写时的心智顺序"排：先起名，再决定它属于什么、有几件，最后才是有没有时间信息、放哪里。开关控制的隐藏行必须在纸面上就画出来，因为"关闭开关时隐藏日期行"是交互状态，不是样式细节。取消和保存都放顶部工具栏，用户随时可能想跑；不放在底部是因为底部会被键盘顶起来。

```
空状态（列表无数据 / 搜索无结果）        状态变化：不是一个空白页

+--------------------------------------+
| 物品                          [+]      |
+--------------------------------------+
|                                        |
|           [ 大号系统符号 ]             |  P1 用 .largeTitle 的 2 到 3 倍
|                                        |
|           还没有任何物品               |  P1 一句话说明现状
|       点右上角的加号添加第一件         |  P2 一句话给出下一步
|                                        |
|           [  添加物品  ]               |  P1 主按钮，最少 44 点高
+--------------------------------------+
```

空状态必须单独画，因为它最容易在写代码时被忘掉，最后交付的是一个纯白页面，用户会以为 App 卡住了。这条线框里最重要的是那个按钮：它和右上角的加号做同一件事。

画完线框再检查三件事：这一页有没有出现两个同等大小的按钮、有没有哪块内容在纸上就写不出具体内容、有没有哪个可点区域小于 44 点。这三个问题在纸上改是几秒钟的事。

2.5 从线框到 SwiftUI 代码的翻译对照表

线框画完之后，写代码基本上是查表。左边是线框上的画法，中间是 SwiftUI 的写法，右边是新手最常踩的坑。

线框里画的东西	对应的 SwiftUI 写法	常见坑
纵向排列的若干块	VStack(alignment: .leading, spacing: 8)	忘了 alignment，块内文字左边缘对不齐
横向并排的图标与文字	HStack(spacing: 8)，图标先固定尺寸	图片尺寸不固定，文字起点每行都不一样
可滚动列表	List，或 ScrollView 加 LazyVStack	用 VStack 硬堆，数据一多就卡且不能滚
分组标题	Section，或 Form 里的自动分组	用 Text 假装分组标题，滚动时不会吸顶
卡片	VStack 加 padding 加 background 加圆角	直接描边代替背景色差，页面全是线
网格	LazyVGrid 加 GridItem(.adaptive(minimum: 150))	用 List 做网格，一列一项，排不出两列
顶部标题	navigationTitle	自己在正文顶部写标题，滚动时位置会跑
右上角的加号	toolbar 里的 ToolbarItem	每页自己画按钮，位置和图标各页不一致
底部页签	TabView 加 Tab	用 HStack 自己画底部栏，点击不切换
从下浮出的表单	.sheet(isPresented:) 或 .sheet(item:)	用 push 代替 sheet，取消没有合适出口
危险操作的确认	.alert 加 isPresented 绑定	直接删不确认，用户误触就丢数据
搜索框	.searchable(text: $keyword)	用 TextField 当搜索，没有清除和取消
状态色文字或角标	.foregroundStyle(.orange)，或 Capsule	把状态色当装饰，一屏出现四种颜色
开关控制的隐藏行	if showExpiry 加 DatePicker	用 opacity 藏起来，位置还占着，留空洞
空状态	ContentUnavailableView，或自己拼 VStack	没有空状态分支，无数据时只剩白屏
缩略图	AsyncImage，固定 56 x 56 并 .clipped()	不裁剪，长图把行高撑成两倍

这张表里的每一行都对应第 1 章的一条规范，取值不需要再讨论：间距 8、12、16、24，圆角 10 和 16，缩略图 56 x 56，可点区域 44 x 44。写代码时只要出现表里没有的数值，就停下来说清楚它属于哪一档。

最后是落地顺序：先按线框把 `Section` 和 `VStack` 的骨架搭出来，每块只放一个 `Text("占位")`，确认结构对了再填内容和样式。这样出问题时你只需要判断"是结构错了还是样式错了"，而这两类问题的排查方法完全不同。

第 3 章 SwiftUI 布局与样式的完整武器库

这一章是字典，不是教程。写界面卡住的时候回来查：布局怎么排、修饰符什么顺序、间距取多少、控件怎么写。所有数值都来自本项目的设计规范，照抄即可，不要临场改数字。

3.1 布局的两个核心概念

一句话结论：SwiftUI 布局只有一条规则，父视图向子视图**提议**一个尺寸，子视图**自己决定**最终尺寸并回报给父视图。

这条规则解释了你遇到的大部分困惑。父视图的提议只是建议，子视图可以拒绝。`Text` 的脾气是"我要刚好装下我的字"，`Color` 和 `Rectangle` 的脾气是"给我多少我占多少"，`Image` 两者都可能。所谓布局，就是这一串提议和回报在视图树上从外往内、再从内往外走了一遍。

```
struct ProposalDemo: View {
    var body: some View {
        // 外层 HStack 把宽度平均提议给两个 Text，但 Text 只按自己的文字宽度取用
        HStack {
            Text("短")
            Text("这是一句很长很长的标题文字")
        }
        .font(.body)
        .padding(16)                                    // 页面左右边距 16
    }
}
```

这段代码怎么读：`HStack` 先算出自己有多少可用宽度，再把它分给两个子视图。第一个 `Text` 只需要一个字的宽度，就把剩下的宽度还了回去；第二个 `Text` 需要多少就拿多少。所以两个 `Text` 宽度不一样，不是 bug，也不是分配不均。如果你希望它们各占一半，必须用 `frame(maxWidth: .infinity)` 明确要求。

想看清提议关系，可以让子视图把收到的提议画出来：

```
struct WidthReader: View {
    var body: some View {
        Text("我把可用宽度撑满")
            .frame(maxWidth: .infinity)                 // 接受父视图的全部宽度提议
            .background(.quaternary)                    // 用背景色看到实际占位
    }
}
```

frame 的三种用法

`frame` 是你唯一需要精确记住的尺寸工具，它有三种用法，覆盖了九成的排版需求。

```
// 1. 固定尺寸：精确控制一个元素，比如列表缩略图
Image(systemName: "shippingbox")
    .resizable()
    .scaledToFill()
    .frame(width: 56, height: 56)                  // 缩略图 56 x 56
    .clipShape(.rect(cornerRadius: 10))            // 缩略图圆角 10

// 2. maxWidth 撑满：让一个元素吃掉所有剩余宽度，而不是定死一个数字
Text("标题")
    .frame(maxWidth: .infinity, alignment: .leading)   // 撑满并把文字靠左

// 3. minHeight 保底：高度至少这么高，内容多时可以变高
Button("保存") { save() }
    .frame(minWidth: 44, minHeight: 44)            // 可点区域最小 44 x 44
```

读法：`frame` 里的宽高参数传入的是确定值，`maxWidth` 和 `minHeight` 传入的是边界条件。视图先按自己的理想尺寸算一遍，再被这些边界条件约束：`maxWidth: .infinity` 表示可以长到无限大，于是它扩张到父视图允许的宽度；`minHeight: 44` 表示不许低于 44，于是矮的按钮被顶到 44。

有一个坑要记住：`frame` 只改变占位和对齐方式，不改变子视图的内容尺寸。如果里面放的是 `Image`，图片本身不会自动缩放，必须配 `resizable()`。反过来，把 `frame(maxWidth: .infinity)` 加在 `Text` 上，文字的行宽变成了整屏宽，长文本的换行位置也会跟着变，这正是我们想要的效果。

```
struct CardRow: View {
    var body: some View {
        HStack(spacing: 8) {                       // 图标与文字之间 8
            Image(systemName: "shippingbox")
                .frame(width: 44, height: 44)      // 图标本身可以小，可点区域要够 44
            VStack(alignment: .leading, spacing: 4) {
                Text("移动硬盘")
                    .font(.body)                   // 列表主文本用 .body
                Text("数量 2")
                    .font(.footnote)               // 次要信息用 .footnote
                    .foregroundStyle(.secondary)
            }
            .frame(maxWidth: .infinity, alignment: .leading)   // 中间整块撑满，把右侧推开
        }
        .padding(16)                               // 卡片内部留白 16
    }
}
```

这段代码里有三个 `frame`，各司其职：第一个固定图标占位，第二个用 `maxWidth: .infinity` 把中间的标题区撑开，第三个是更外层的 `padding` 提供呼吸空间。写完一个界面后如果你觉得两边挤在一起，先检查是不是漏了 `maxWidth` 的那一层。

> 结论：布局出问题时，先问三个问题，父视图提议了多少、子视图愿意要多少、哪里用 `frame` 改写了尺寸。九成的位置不对都能这样定位。

三种 frame 怎么选

写完界面之后回头看，尺寸相关的写法只有三种，选择顺序也固定：

1. 能用语义尺寸就不用数字。图标用 `.font(.title3)`，缩略图按规范的 56 x 56，别自己发明 58。
2. 需要撑满或对齐就用 `maxWidth: .infinity` 加 `alignment`，不要用固定的宽数字去凑。
3. 只有确定是死尺寸的元素才写 `width` 和 `height`，比如缩略图、头像、图标按钮。

最常见的三个误用要认得出来。第一个是到处写 `frame(width: 375)`，把屏幕宽度写死，换到 iPad 或者分屏直接错位。第二个是给 `VStack` 加 `frame(maxHeight: .infinity)` 想让内容垂直居中，结果把后面的兄弟视图全挤出屏幕，正确做法是在外层套 `Spacer()` 或者对单个元素设对齐。第三个是以为 `frame` 会缩放内容，于是给 `Image` 加了固定尺寸却没加 `resizable()`，图片依然按原始大小绘制，只是被裁掉了。

```
// 想让一组内容垂直居中：用 Spacer 分配剩余空间，而不是给容器设无限高度
VStack {
    Spacer()
    ContentCard()
    Spacer()
}
```

3.2 容器与排列

一句话结论：容器负责决定子视图怎么排，选容器比调参数重要一百倍。

VStack 与 HStack

垂直堆叠与水平堆叠，最常用的两个容器，参数只有 `spacing` 和 `alignment`。

```
VStack(alignment: .leading, spacing: 12) {        // 卡片与卡片之间 12
    Text("详情").font(.headline)                  // 区块标题用 .headline
    Text("放在书桌第二层抽屉").font(.body)
}
```

什么时候用：一组信息从上到下读完，用 `VStack`；同一行的图标加文字加箭头，用 `HStack`。注意 `alignment` 对 `VStack` 是指水平方向的对齐，对 `HStack` 是指垂直方向的对齐，这一点新手最容易搞反。

ZStack

子视图叠在一起，后写的盖在上面，用来做背景、角标、自定义按钮。

```
ZStack(alignment: .topTrailing) {
    RoundedRectangle(cornerRadius: 16)            // 卡片圆角 16
        .fill(Color(.secondarySystemGroupedBackground))
    Image(systemName: "star.fill")                // 收藏角标叠在右上角
        .foregroundStyle(.orange)
        .padding(12)
}
.frame(height: 120)
```

什么时候用：需要盖的关系时用，比如卡片右上角的收藏角标、头像上的在线小圆点。纯排列不要用 `ZStack`，它会让视图互相遮挡，调试成本比 `VStack` 高。

Spacer 与 Divider

`Spacer` 是唯一的弹性空白，`Divider` 是唯一被推荐的分隔线。

```
HStack {
    Text("名称")
    Spacer(minLength: 8)                          // 把右侧内容推到最右，最少留 8
    Text("移动硬盘").foregroundStyle(.secondary)
}
Divider()                                          // 一条 1 点的系统分隔线
```

什么时候用：同一行里左边标题右边数值，中间用 `Spacer`；列表或表单里需要视觉分段，用 `Divider`。不要用空 `Text("")` 或者一大串空格来占位。

Grid 与 GridRow

真正的二维表格布局，iOS 16 起可用，能让同一列的宽度在所有行之间自动对齐。

```
Grid(alignment: .leading, horizontalSpacing: 12, verticalSpacing: 10) {
    GridRow {
        Text("数量").foregroundStyle(.secondary)
        Text("2").gridColumnAlignment(.trailing)   // 这一列统一靠右
    }
    GridRow {
        Text("位置").foregroundStyle(.secondary)
        Text("书桌第二层")
    }
}
```

什么时候用：数据是字段名加字段值的键值对，而且要求多行之间的数值位置对齐，就用 `Grid`。如果只是每行一个卡片，那是 `VStack` 的活。

LazyVGrid 与 LazyHGrid

网格版 `List`：按列数或固定列宽排布，内容多时才创建视图，适合卡片墙。

```
// 自适应列宽：每列至少 150 宽，能塞几列塞几列，间距 12
let columns = [GridItem(.adaptive(minimum: 150), spacing: 12)]

ScrollView {
    LazyVGrid(columns: columns, spacing: 12) {    // 卡片与卡片之间 12
        ForEach(items) { item in
            ItemCard(item: item)
        }
    }
    .padding(16)                                  // 页面左右边距 16
}
```

什么时候用：图片墙、物品网格、标签墙。`GridItem` 的三种写法各有用途：`.adaptive(minimum:)` 自动适配列数，`.flexible()` 平均分，`.fixed(120)` 固定宽。横向网格用 `LazyHGrid(rows:)`，通常配合横向 `ScrollView`。

ScrollView 两个方向

```
ScrollView(.vertical, showsIndicators: false) {   // 纵向滚动
    ContentColumn()
}

ScrollView(.horizontal, showsIndicators: false) { // 横向滚动，常用于横向卡片列表
    LazyHStack(spacing: 12) {
        ForEach(tags, id: \.self) { tag in TagChip(text: tag) }
    }
    .padding(.horizontal, 16)
}
```

什么时候用：内容长度不确定就套 `ScrollView`。注意两点：`ScrollView` 本身没有尺寸，必须由外部给它高度；横向滚动里的子视图一定要给出明确宽度，否则它算不出内容总宽。

List、Form 与 Section

`List` 是带系统样式、复用和滑动手势的滚动列表；`Form` 是 `List` 的设置页皮肤；`Section` 给它们分组。

```
List {
    Section("基本信息") {
        LabeledContent("数量", value: "\(item.quantity)")
        LabeledContent("位置", value: item.location ?? "未填写")
    }
    Section("操作") {
        Button("删除", role: .destructive) { delete() }
    }
}
```

什么时候用：数据行超过一屏、需要滑动删除或选中、或者做设置类界面，用 `List` 或 `Form`。如果你只是想排几行不滚动的信息，别用 `List`，它自带的内边距和分隔线会跟你打架，用 `VStack`。

Group

`Group` 不改变布局，只是把多个视图打包成一个，用来批量加修饰符或让 `if` 的两个分支返回同一种类型。

```
Group {
    Text("第一段")
    Text("第二段")
}
.font(.body)                                        // 一次作用到组内所有 Text
.foregroundStyle(.secondary)
```

什么时候用：多个视图要加同一个修饰符，或者条件分支需要统一类型时。记住 `Group` 不是容器，它不会让子视图并排或堆叠。

汇总表

容器	什么时候用	注意点
VStack	一组信息从上到下读	alignment 是水平方向对齐
HStack	一行内的图标、文字、数值	间距用 spacing，不要塞空 Text
ZStack	背景、角标、叠加层	后写的在上层，别用它做纯排列
Spacer	把同行内容推到两端	用 minLength 保底，别用空格占位
Divider	需要一条视觉分隔线	List 与 Form 自带分隔线，不用手加
Grid 与 GridRow	键值对要对齐成二维表	列宽由整列决定，用 gridColumnAlignment 调
LazyVGrid 与 LazyHGrid	卡片墙、网格	外层必须有 ScrollView 和明确的列定义
ScrollView	内容长度不确定	横向滚动要给子视图明确宽度
List	长列表、可滑动、可选中	默认样式重，纯展示别用
Form	设置页、录入页	就是 List 的一种样式
Section	给 List 或 Form 分组	可以带 header 和 footer
Group	批量加修饰符	它不做布局，只做打包

容器怎么嵌套

真实的界面都是嵌套出来的，嵌套的原则只有一条：先按阅读顺序分成几个块，再把每个块拆成行，最后才考虑用不用网格。一个详情页的骨架大致是这样：

```
ScrollView {
    VStack(alignment: .leading, spacing: 24) {     // 区块与区块之间 24
        HeaderBlock()                              // 第一块：头图加标题
        VStack(alignment: .leading, spacing: 12) { // 第二块：信息列表，行间距 12
            InfoRow(label: "数量", value: "2")
            InfoRow(label: "位置", value: "书桌第二层")
        }
        .padding(16)                               // 卡片内部留白 16
        .background(Color(.secondarySystemGroupedBackground),
                    in: .rect(cornerRadius: 16))   // 卡片圆角 16
        ActionBar()                                // 第三块：操作按钮
    }
    .padding(16)                                   // 页面左右边距 16
}
```

读法：最外层 `ScrollView` 负责滚动，`VStack` 负责分块，内层 `VStack` 负责一组行，`InfoRow` 内部才是一个 `HStack`。层级不要超过四层，超过就说明该抽成独立的子视图了。抽出去之后每个视图只关心自己那一小块，预览和复用都会容易很多。

还有一条经验：`List` 和 `ScrollView` 不要互相嵌套。`List` 内部已经是滚动容器，再包一层 `ScrollView` 会导致高度计算失准、滚动冲突。如果需要在列表上方放一段不滚动的头图，用 `Section` 的 header，或者让整页一起滚动，两种都可以，但不要嵌套。

3.3 修饰符的顺序为什么重要

一句话结论：SwiftUI 的修饰符是**从内到外**一层层包上去的，先写的在里面，后写的在外面，顺序换了结果就换。

修饰符不是给这个视图设置属性，而是用这个视图造出一个新视图。`Text("标题").padding(16).background(.blue)` 的意思是：先造一个文字，再造一个文字加内边距的包装，最后造一个蓝色背景贴在包装外面。看下面两个例子，它们只差一行顺序。

```
// 写法 A：先 padding 再 background，蓝色块很大，文字周围有 16 点蓝色
Text("标签")
    .padding(16)
    .background(.blue)

// 写法 B：先 background 再 padding，蓝色紧贴文字，外面是 16 点透明留白
Text("标签")
    .background(.blue)
    .padding(16)
```

结果差异：A 的蓝色区域是文字宽度加 32，因为你先把文字撑大了，再给这个更大的东西上色；B 的蓝色区域只有文字那么宽，因为你给文字本身上了色，之后才在外面留白。做标签胶囊、筛选按钮时，几乎总是要 A 的写法。

同样的道理适用于 `frame` 与 `background`：

```
// 写法 A：background 在内，只有文字本身有底色，frame 撑开的区域是透明的
Text("标题")
    .background(.blue)
    .frame(maxWidth: .infinity, alignment: .leading)

// 写法 B：frame 在内、background 在外，整行都是蓝色，文字靠左
Text("标题")
    .frame(maxWidth: .infinity, alignment: .leading)
    .background(.blue)
```

读法：background 永远贴着它前面已经算完尺寸的那个视图。想要整行背景色铺满，就先把宽度撑开再上色；想要一小块高亮跟着文字走，就先上色再撑开。列表高亮的错位感，多半就是这里写反了。

再看一个更隐蔽的组合，clipShape 与 shadow：

```
// 正确：先裁圆角，再投影，阴影会跟着圆角形状走
Image(systemName: "photo")
    .frame(width: 220, height: 220)
    .clipShape(.rect(cornerRadius: 16))           // 详情页头图圆角 16
    .shadow(color: .black.opacity(0.06), radius: 8, y: 2)

// 错误：先投影再裁圆角，方形阴影被裁掉一半，看起来脏
Image(systemName: "photo")
    .shadow(color: .black.opacity(0.06), radius: 8, y: 2)
    .clipShape(.rect(cornerRadius: 16))
```

从内到外的记忆方法：把这个视图的链条念一遍，问自己每一步现在处理的是什么东西。上色、裁剪、投影这些视觉修饰符，作用对象永远是它左边那一整串已经成型的视图。于是有三条可以背下来的习惯：

- 内容和内边距在内，背景和圆角在外，顺序是内容、padding、background、clipShape。
- 尺寸和位置决定占多大，视觉修饰决定长什么样，视觉修饰通常要放在尺寸之后。
- 交互修饰符（onTapGesture、contentShape）放在最后，它作用于最终成型的那块区域。

```
struct TagChip: View {
    let text: String

    var body: some View {
        Text(text)
            .font(.caption)                       // 标签用 .caption
            .foregroundStyle(.secondary)
            .padding(.horizontal, 8)              // 图标与文字之间 8
            .padding(.vertical, 4)
            .background(.quaternary, in: .capsule)
            .contentShape(.capsule)               // 让整个胶囊都可点，而不只是文字
            .onTapGesture { select() }
    }
}
```

这段代码的链条是文字、缩小字号、变灰、横向内边距、纵向内边距、胶囊背景、可点区域、点击事件，每一步都建立在前面所有步骤的结果之上。写的时候从头到尾念一遍，顺序就不会错。

一行速查表，按这个顺序写基本不会错：

想要的视觉效果	正确顺序
整行背景色铺满	frame 在前，background 在后
小块高亮跟着文字	background 在前，frame 在后
胶囊标签	padding 在前，background 加 in 形状在后
圆角加投影	clipShape 在前，shadow 在后
圆角加描边	填充和 overlay 描边在同一层，框架在最外
整块可点击	所有视觉修饰之后，加 contentShape 再挂手势

3.4 间距、内边距与留白

一句话结论：留白不是剩下多少算多少，而是从设计规范里取固定值，本项目只用 4 的倍数。

间距刻度表，本章所有 `padding`、`spacing`、`VStack` 参数都从这里取：

位置	取值
图标与文字之间	8
卡片内部留白	12 或 16
列表行上下留白	10
页面左右边距	16
卡片与卡片之间	12
区块与区块之间	24

padding 的各种写法

```
Text("四周都留 16")
    .padding(16)                                    // 四边相同

Text("只留左右")
    .padding(.horizontal, 16)                       // 页面左右边距 16

Text("只留上下")
    .padding(.vertical, 10)                         // 列表行上下留白 10

Text("左侧只留 8")
    .padding(.leading, 8)                           // 图标与文字之间 8

Text("上 24 下 12")
    .padding(.top, 24)
    .padding(.bottom, 12)
```

优先用 `.horizontal` 和 `.vertical`，实在不对称再拆成单边。连续写多个 `padding` 是允许的，它们会累加：`.padding(.horizontal, 16)` 再加 `.padding(.leading, 8)` 等于左边 24、右边 16，这种写法可读性差，只在真的需要时才用。

spacing 用在哪

`spacing` 是容器的属性，写在 `VStack`、`HStack`、`LazyVGrid` 的初始化参数里，控制子视图之间的距离。

```
VStack(alignment: .leading, spacing: 24) {        // 区块与区块之间 24
    SectionHeader(title: "基本信息")
    VStack(alignment: .leading, spacing: 12) {    // 卡片内部两行之间 12
        InfoRow(label: "数量", value: "2")
        InfoRow(label: "位置", value: "书桌第二层")
    }
}
.padding(16)                                      // 卡片内部留白 16
```

一个经验规则：`spacing` 管元素之间，`padding` 管元素与边界之间。同一组的两行用 8 到 12，不同区块之间用 24，页面边界用 16。

alignment 对齐

`alignment` 决定子视图尺寸不齐时靠在哪一边。`VStack(alignment: .leading)` 让所有子视图左对齐，`HStack(alignment: .firstTextBaseline)` 让不同字号的文字基线对齐。

```
HStack(alignment: .firstTextBaseline, spacing: 8) {
    Text("¥ 199").font(.title3)                   // 区块标题用 .title3
    Text("含税").font(.caption)                    // 角标用 .caption
    Spacer()
}
```

什么时候用 `.firstTextBaseline`：一行里同时有大字号和小字号文字，用默认的 `.center` 会让小字浮在中间，基线对齐才像排版。图标加文字的组合通常也用基线对齐。

safeArea 怎么处理

默认情况下，视图被限制在安全区内，灵动岛和底部 Home 指示条的位置不会被内容占用。给滚动内容统一留边距时，优先用 `contentMargins`，不要自己猜数字。

```
ScrollView {
    ContentColumn()
}
.contentMargins(.horizontal, 16, for: .scrollContent)   // 滚动内容统一左右 16
```

`ignoresSafeArea` 只在这三种情况用：整屏背景色、沉浸式大图、自定义全屏画布。反过来，如果你发现列表最后一项被 Home 指示条挡住，不要用 `ignoresSafeArea` 去顶，应该检查是否误用了 `.ignoresSafeArea(.container, edges: .bottom)`。

```
ZStack {
    Color(.systemGroupedBackground)              // 页面底色
        .ignoresSafeArea()                       // 只有背景色扩展到全屏
    VStack(spacing: 12) {
        Card()
    }
    .padding(16)                                 // 内容仍留在安全区内
}
```

读法：先铺一层忽略安全区的纯色背景，再把内容正常放进 `ZStack`。这是整屏背景加安全内容的标准写法，比在每张卡片上手动调边距可靠得多。需要给滚动内容额外留安全区空间时，用 `.safeAreaPadding()`，不要自己算数字。

间距取值速查

拿到一个界面不知道写几，按这张表对号入座，全部来自设计规范：

你正在排的东西	取值
图标紧挨文字	8
卡片里的一行和下一行	12
卡片四周到内容	16
列表行的上下	10
页面内容到屏幕边缘	16
两张卡片之间	12
两个功能区块之间	24

有两个容易被忽略的硬规则。可点区域最小 44 x 44，图标本身可以只有 20 点，但要给它加 `.frame(minWidth: 44, minHeight: 44)` 或者用 `.contentShape(.rect)` 把响应区域撑开，否则手指按不准。另一个是间距只用 4 的倍数，不要出现 13、15、18 这样的值，它们是排版看起来不整齐的根源。

3.5 背景、圆角、边框与阴影

一句话结论：层级关系优先靠背景色差、圆角和间距表达，阴影是最后手段，本项目几乎不用。

background 的五种形态

```
// 1. 纯色
Text("页面底色块").background(Color(.systemGroupedBackground))

// 2. 材质：能透出后面内容的毛玻璃，适合浮层和工具栏
Text("浮层").padding(12).background(.regularMaterial, in: .rect(cornerRadius: 16))

// 3. 渐变：用色相接近的两个颜色，不要做彩虹色
RoundedRectangle(cornerRadius: 16)
    .fill(LinearGradient(colors: [.blue, .cyan],
                         startPoint: .topLeading,
                         endPoint: .bottomTrailing))

// 4. 形状背景：一次完成上色加裁剪，不用再写 clipShape
Text("胶囊")
    .padding(.horizontal, 12)
    .background(.tint, in: .capsule)

// 5. 语义背景：跟随深浅色自动切换
Text("卡片").padding(16).background(Color(.secondarySystemGroupedBackground))
```

`background(_:in:)` 是推荐写法，它一次完成上色加裁剪，比 `background` 加 `clipShape` 少一层，也不容易写错顺序。材质 `.regularMaterial` 和 `.thinMaterial` 用在弹出菜单、悬浮工具栏、半透明卡片上；普通卡片不要用材质，用语义背景色，因为材质会透出底层内容，列表滚动时会花。

圆角：三种写法的区别

```
// 推荐：可随内容尺寸变化，支持连续圆角，新代码默认用它
.clipShape(.rect(cornerRadius: 16))

// 可用：效果等价，写起来长一点，需要显式声明连续圆角时用
.clipShape(RoundedRectangle(cornerRadius: 16, style: .continuous))

// .cornerRadius(16) 已过时，不要用：不能做连续圆角，也没法和别的形状组合
```

`.rect(cornerRadius:)` 是 iOS 17 引入的形状简写，可以和其他形状一起放进 `clipShape`、`background(_:in:)`、`containerShape`，推荐统一用它。圆角取值受一个界面最多两种的约束，按元素类型取：

元素	取值
列表缩略图	10
卡片、大图	16
按钮、输入框	10，或者整条胶囊形

overlay 加描边

描边用 `overlay` 叠一个 `stroke`，而不是 `border`。`border` 只能画直角矩形，圆角卡片上会露出一个方框。

```
RoundedRectangle(cornerRadius: 16)
    .fill(Color(.secondarySystemGroupedBackground))
    .overlay {
        RoundedRectangle(cornerRadius: 16)
            .strokeBorder(.quaternary, lineWidth: 1)   // 语义色，深浅色模式都正常
    }
    .frame(height: 120)
```

需要描边不撑大外框时用 `strokeBorder` 而不是 `stroke`，它把线宽画在形状内部。选中态的高亮也是这个套路：把 `.strokeBorder(.tint, lineWidth: 2)` 加在原有的圆角形状上。

shadow 的正确参数

```
Card()
    .shadow(color: .black.opacity(0.06), radius: 8, x: 0, y: 2)
```

四个参数的含义：`color` 控制浓淡，用极低的透明度；`radius` 是模糊半径，越大越飘；`x` 和 `y` 是偏移，`y: 2` 表示光从上方来。本项目只允许这一种极轻的阴影，不要用默认的 `.shadow(radius: 10)`，它在深色模式下会变成一团黑边。

重要原则：**多层级的界面优先用分组背景，而不是阴影。** 卡片放在 `Color(.systemGroupedBackground)` 的页面上，卡片本身用 `Color(.secondarySystemGroupedBackground)`，这个色差已经足够表达层级。再加阴影只会让界面显得脏。阴影只留给真正浮起来的东西：悬浮按钮、弹出面板、正在拖动中的卡片。

背景类型怎么选

背景类型	什么时候用
Color 语义色	页面底色、卡片底色，日常首选
Material 材质	弹出层、悬浮工具栏、需要透出下层的浮层
渐变	头图、空状态插画背景，同色系两个颜色就够
形状背景	标签、徽章、胶囊按钮

判断方法是问一句：这块背景下面有没有内容需要被看见。不需要被看见就用不透明语义色，需要就考虑材质或半透明。渐变只用在视觉焦点上，一屏最多一处，用太多界面会显得廉价。

3.6 文字排版

一句话结论：字号一律用系统的语义字号，绝不写死 `.system(size:)`，因为写死之后就不再跟随用户的文字大小设置。

语义字号阶梯，从大到小：

用途	SwiftUI 写法
页面主标题	.largeTitle
区块标题	.title3 或 .headline
正文与列表主文本	.body
次要信息	.subheadline 或 .footnote
时间、标签、角标	.caption 或 .caption2

规则：一个界面最多出现 4 种字号，标题和正文之间至少差一个档位。`.headline` 和 `.body` 字号相同但字重不同，用它做同一段里的强调小标题很合适。

```
VStack(alignment: .leading, spacing: 8) {
    Text("移动硬盘")                                  // 正文与列表主文本
        .font(.body)
    Text("最后更新于 3 天前")                          // 次要信息
        .font(.footnote)
        .foregroundStyle(.secondary)
    Text("已过期")                                    // 角标
        .font(.caption2)
        .foregroundStyle(.red)                        // 状态色只用来表达状态
}
```

不要写 `.font(.system(size: 14))` 的原因只有一条，但足够致命：用户的系统文字大小调大之后，你的 14 点文字一动不动，旁边用语义字号的文字全部变大，整个界面就散架了。辅助功能里更大文字开到最大档时，写死尺寸的界面基本不可用。

字重与字形

```
Text("重点").fontWeight(.semibold)                    // 在语义字号上加字重
Text("代码片段").fontDesign(.monospaced)              // 等宽，适合展示标识符
Text("打卡").fontDesign(.rounded)                     // 圆体，适合轻松的数字展示
```

`fontWeight` 只调整粗细，不改变尺寸，所以可以和任何语义字号组合。`fontDesign` 改变字形家族，只在确实需要区分信息类型时用，同一个界面里不要混用超过两种。

行距、对齐与截断

```
Text("这是一段较长的说明文字，讲清楚这个物品放在哪里、为什么放在那里，以及什么时候需要检查一次。")
    .font(.body)
    .lineSpacing(4)                                    // 行与行之间额外 4 点，长段落才需要
    .multilineTextAlignment(.leading)                  // 多行文本左对齐
    .lineLimit(3)                                      // 最多三行，超出截断
    .truncationMode(.tail)                             // 省略号出现在结尾
```

单行文本不需要 `lineSpacing`，多行长段落加 2 到 4 点就够，加太多会散。

`multilineTextAlignment` 默认就是 `.leading`，只有标题居中时才写 `.center`，中文段落不要用两端对齐。`lineLimit` 用在卡片摘要上，避免长短不一的卡片把网格撑破。

minimumScaleFactor

```
Text("¥ 1,299.00")
    .font(.title3)
    .lineLimit(1)
    .minimumScaleFactor(0.7)                           // 空间不够时可以缩到 70%
```

什么时候用：数值长度不可控、又必须一行显示完的地方，比如价格、大号统计数字。

`minimumScaleFactor` 是保底缩小，它优先缩字号而不是截断。不要拿它当万能药，如果一段文字缩到 0.7 还放不下，说明布局本身该改，而不是继续调这个值。

字号怎么选

拿到一段文字，先问它在界面里扮演什么角色，再查表。判断不了的时候有两个默认答案：正文一律 `.body`，说明和辅助信息一律 `.footnote` 加 `.secondary`。这两个组合覆盖了大部分场景，先这么写，之后再按层次调整。

`.font` 可以加在容器上，让整棵子树继承同一个字号，这比给每个 `Text` 单独写省事，也保证了不会出现第四种、第五种字号：

```
VStack(alignment: .leading, spacing: 12) {
    Text("基本信息").font(.headline)                  // 区块标题，唯一一处变化
    Text("移动硬盘")
    Text("数量 2")
    Text("书桌第二层")
}
.font(.body)                                           // 其余全部继承正文
```

读法：`.font(.body)` 挂在外层 `VStack` 上，里面没有单独设字号的 `Text` 全部继承它。只有需要区分层次的区块标题才单独覆盖成 `.headline`。这个写法让界面里的字号种类自然收敛到两三种，也符合一个界面最多 4 种字号的规则。

3.7 颜色系统

一句话结论：只用语义色和角色色，永远不要写 `Color.white` 当背景、`Color.black` 当文字。

`Color` 是具体颜色，`ShapeStyle` 是角色的表达方式。SwiftUI 里大量 API 接收的是 `ShapeStyle`，所以你可以直接传 `.primary`、`.tint`、`.quaternary` 这类语义值，也可以传 `Color.blue`。日常写法是优先传语义值，需要固定品牌色时才传 `Color`。

文字层级：primary、secondary、tertiary

角色	用法
主文字	.primary
次要文字	.secondary
更弱的说明	.tertiary

```
VStack(alignment: .leading, spacing: 4) {
    Text("移动硬盘").font(.body).foregroundStyle(.primary)
    Text("数量 2").font(.subheadline).foregroundStyle(.secondary)
    Text("未设置到期提醒").font(.caption).foregroundStyle(.tertiary)
}
```

这三个不是灰度的三档，而是随深浅色模式自动翻转的角色色：浅色下 `.primary` 是近黑，深色下它变成近白。`foregroundStyle` 是现代写法，能一次作用于整个视图树；`foregroundColor` 已过时，不要用，它只接受 `Color`，表达不了 `.secondary` 这类层级样式。

页面底色与卡片底色

```
ScrollView {
    VStack(spacing: 12) {                          // 卡片与卡片之间 12
        Card()
    }
    .padding(16)                                   // 页面左右边距 16
}
.background(Color(.systemGroupedBackground))       // 页面底色

// 卡片内部
RoundedRectangle(cornerRadius: 16)
    .fill(Color(.secondarySystemGroupedBackground))   // 卡片底色
```

`Color(.systemGroupedBackground)` 是带括号加 `.` 前缀的写法，它借用 UIKit 的系统色常量，会正确跟随浅色和深色模式。这两个值必须成对使用：页面用 grouped 版本，卡片用 secondary 版本，色差刚好够看出层级。不要自己写 `Color(white: 0.96)` 这种近似色，深色模式下它一定是错的。

强调色：tint 与它的旧名字 accentColor 已过时

```
// 在根视图统一设置一次，全 App 的按钮、开关、选中态都会跟随
WindowGroup {
    RootView()
        .tint(.blue)
}
```

`.tint` 是 iOS 15 起的推荐写法，`.accentColor` 是它的旧名字，已过时，不要再用。规则是一个界面最多 1 个强调色加 3 个状态色，强调色只在根视图设置一次，不要在每个按钮上重复写 `.tint`，否则改主题时要改一百个地方。

HierarchicalShapeStyle 与状态色

`.primary`、`.secondary`、`.tertiary`、`.quaternary` 以及它们的 `.fill` 变体，统称层级样式（HierarchicalShapeStyle），它的作用是把当前前景色按透明度分层，而不是指定某个色相。所以 `.foregroundStyle(.blue)` 之后再用 `.secondary`，得到的是蓝色的弱化版本，这个组合非常省事。

状态	颜色
正常	.green
临期	.orange
过期	.red
已归档	.secondary

```
// 状态色只用来表达状态，不要拿它当装饰
Circle()
    .fill(item.expiryState.color)
    .frame(width: 8, height: 8)                    // 状态小圆点

extension ExpiryState {
    var color: Color {
        switch self {
        case .none: .secondary                      // 无日期或已归档用次级色
        case .safe: .green
        case .soon: .orange                         // 30 天内
        case .expired: .red
        }
    }
}
```

这条规则要严格执行：绿色只能表示正常，橙色只能表示临期，红色只能表示过期或危险。把红色拿来当按钮装饰，用户会以为那个按钮有风险。深色模式下 `.systemGroupedBackground` 会变成近黑，`.primary` 会变成白色，这就是永远不要写死黑白的原因。

两个查错习惯

第一个习惯是把界面切到深色模式看一遍。写死的 `Color.white`、`Color.black`、`Color(white:)` 在这一步会全部暴露，而 `.primary`、`.secondary`、`Color(.systemGroupedBackground)` 都会自动翻转，不会出问题。

第二个习惯是数一数界面里的颜色。规则是一个界面最多 1 个强调色加 3 个状态色，超过就说明有颜色在当装饰用。装饰性的颜色诉求，用 `.quaternary` 这类层级样式或者背景色差来解决，不要新增色相。

3.8 图片与 SF Symbols

一句话结论：图标一律用 SF Symbols 加 `Image(systemName:)`，照片用 `Image` 加 `resizable` 加 `aspectRatio` 加 `clipped` 三件套。

系统符号

```
Image(systemName: "shippingbox")                    // 线性图标，跟随文字颜色
    .font(.title3)                                  // 用字号控制图标大小，不要写死尺寸
    .foregroundStyle(.secondary)

Image(systemName: "shippingbox.fill")               // 填充版，用于选中态
    .font(.title3)
    .foregroundStyle(.tint)
```

读法：SF Symbols 是矢量字体图标，`Image(systemName:)` 得到的是一个能像文字一样被排版的视图。控制大小用 `.font()`，它会跟随动态字体一起缩放；用 `.frame(width:height:)` 配合 `resizable()` 也能改尺寸，但那样就失去了随文字大小缩放的能力，只有需要精确 56 点缩略图时才这样做。

resizable、aspectRatio 与 clipped

```
// 填满容器并裁掉溢出部分，配合固定尺寸使用
Image("cover")
    .resizable()
    .scaledToFill()                                 // 等比放大直到填满
    .frame(width: 56, height: 56)                   // 缩略图 56 x 56
    .clipped()                                      // 裁掉超出 frame 的部分
    .clipShape(.rect(cornerRadius: 10))             // 缩略图圆角 10

// 完整显示整张图，容器里会留白
Image("cover")
    .resizable()
    .scaledToFit()                                  // 等比缩小直到放得下
    .frame(height: 220)                             // 详情页头图高度 220
```

`scaledToFill` 与 `.clipped()` 必须成对出现。`scaledToFill` 会让图片超出容器边界，视觉上溢出到邻居身上；`.clipped()` 把溢出部分剪掉。少了它，图片会盖住旁边的文字。

`clipped` 与 `clipShape` 的区别也很关键：`clipped` 只按当前 `frame` 的矩形边界裁剪，不做圆角；`clipShape` 按你给的形状裁剪，可以是圆角、圆形、胶囊。两张都要圆角时，用 `clipShape` 一个就够，它同时完成了矩形边界和圆角。需要非矩形的裁剪（比如圆形头像）必须用 `clipShape(.circle)`。

symbolRenderingMode 与 symbolEffect

```
// 多色符号：同色系分层，看起来比单色更精致
Image(systemName: "folder.badge.plus")
    .symbolRenderingMode(.hierarchical)

// 双色符号：主色加辅助色，用于状态表达
Image(systemName: "wifi")
    .symbolRenderingMode(.palette)
    .foregroundStyle(.white, .green)

// 状态切换时的短动画，iOS 17 起可用
Image(systemName: "star.fill")
    .symbolEffect(.bounce, value: item.isFavorite)
```

`symbolRenderingMode` 的四种取值：`.monochrome` 单色（默认），`.hierarchical` 单色分层，`.palette` 多色需要按顺序给前景色，`.multicolor` 使用符号自带的颜色。`.symbolEffect` 是 iOS 17 起的能力，第二个参数 `value` 是触发条件，值变化时播一次动画，非常适合收藏、完成、刷新这类状态反馈。不要给符号加无限循环的持续动画，那会一直耗电。

从 Data 显示图片

数据库里存的是 `Data`，要变成图片显示出来，标准写法是先转成 `UIImage` 再交给 `Image`：

```
import SwiftUI
import UIKit                                        // UIImage 来自 UIKit，必须导入

struct ItemThumbnail: View {
    let imageData: Data?

    var body: some View {
        if let imageData, let uiImage = UIImage(data: imageData) {
            Image(uiImage: uiImage)                 // 用 Image(uiImage:) 包装
                .resizable()
                .scaledToFill()
                .frame(width: 56, height: 56)       // 缩略图 56 x 56
                .clipShape(.rect(cornerRadius: 10))
        } else {
            // 没有图片时给一个占位图标，避免布局塌陷
            Image(systemName: "photo")
                .font(.title3)
                .foregroundStyle(.tertiary)
```

```
                .frame(width: 56, height: 56)
                .background(.quaternary, in: .rect(cornerRadius: 10))
        }
    }
}
```

这段代码怎么读：`UIImage(data:)` 是可选值，可能失败，所以要 `if let` 两层连着解包。`Image(uiImage:)` 是 SwiftUI 提供的初始化器，和 `Image("资源名")`、`Image(systemName:)` 是并列的三种来源。占位分支一定要写，否则没图片的物品在列表里会缩成一个空洞，整行高度都变了。

图标与图片的尺寸规范

场景	规格
列表缩略图	56 x 56，圆角 10，裁剪填充
网格卡片封面	正方形，圆角 16
详情页头图	宽度撑满，高度 220
空状态插画图标	系统符号，字号 .largeTitle 的 2 到 3 倍

空状态那一条是唯一鼓励把图标放很大的地方，写法是 `.font(.system(size: 56))` 之外更好的选择：用 `.imageScale(.large)` 加 `.font(.largeTitle)`，或者直接用 `ContentUnavailableView`，它内部已经按这个规格处理好了。

3.9 常用控件速查

一句话结论：先在这张表里找到控件，再抄对应示例，不要自己用 `Text` 加 `onTapGesture` 造控件。

控件	写法	用在哪	注意点
Button	Button("保存") { save() }	主动作、次要动作	role 用 .destructive 表示危险操作
TextField	TextField("名称", text: $name)	单行输入	必须双向绑定，用 .textFieldStyle 统一外观
SecureField	SecureField("密码", text: $pwd)	密码、密钥	只能单行，输入内容不回显
TextEditor	TextEditor(text: $note)	多行备注	自带白底，需要自己加 .scrollContentBackground(.hidden)
Toggle	Toggle("收藏", isOn: $isFavorite)	开关型设置	标签太长时用 Toggle 加 LabeledContent 拆开

控件	写法	用在哪	注意点
Picker	Picker("分类", selection: $c)	从固定选项里选一个	样式决定形态，记得给每个选项加 tag
DatePicker	DatePicker("到期", selection: $date)	日期与时间	displayedComponents 控制显示到天还是到分
Stepper	Stepper("数量", value: $qty, in: 1...99)	小范围数值增减	范围一定要写，避免出现 0 或负数
Slider	Slider(value: $level, in: 0...1)	连续区间调节	需要精确值时配 step 参数
Menu	Menu("更多") { ... }	收纳次要操作	内容用 Button 组织，别塞进整个页面
Link	Link("官网", destination: url)	打开外部网址	会自动跳到浏览器，不是应用内
Label	Label("数量", systemImage: "number")	图标加文字的标准组合	列表里优先用它，间距系统已经调好
ProgressView	ProgressView(value: 0.4)	进度、耗时任务	无参数版本是不确定进度转圈
Gauge	Gauge(value: 0.7) { Text("容量") }	带刻度的指标展示	视觉比较重，一屏最多一个
List 的 selection	List(selection: $selected) { ... }	多选、批量操作	元素需要可哈希的 id，编辑模式要显式开
.searchable	.searchable(text: $query)	列表顶部搜索	必须挂在 NavigationStack 内的视图上
.swipeActions	.swipeActions { Button(role: .destructive) }	行内快捷操作	只能挂在 List 的行上，左侧操作要加 edge
.refreshable	.refreshable { await load() }	下拉刷新	闭包必须是 async，动作要足够短
.contextMenu	.contextMenu { Button("删除") }	长按菜单	里面放的也要是 Button
.sheet	.sheet(isPresented: $show) { ... }	弹出独立任务页	内容需要自己包 NavigationStack
.alert	.alert("确认", isPresented: $show) { ... }	必须中断的确认	按钮顺序由系统决定，危险动作用 role
.confirmationDialog	.confirmationDialog("选择", isPresented: $show) { ... }	多选项、危险动作	出现在底部，比 alert 更适合选择

五个常用控件的完整示例

按钮的三种形态，重点是危险操作必须标 `role`：

```
struct ActionButtons: View {
    @State private var isFavorite = false

    var body: some View {
        VStack(spacing: 12) {
            // 主按钮：胶囊形，宽度撑满，最小高度 44
            Button("保存修改") { save() }
                .buttonStyle(.borderedProminent)
                .frame(maxWidth: .infinity, minHeight: 44)

            // 次要按钮：只描边，不抢主按钮的视觉
            Button("取消") { cancel() }
                .buttonStyle(.bordered)

            // 危险操作：role 让系统自动用红色表达风险
            Button("删除物品", role: .destructive) { delete() }
                .buttonStyle(.bordered)
        }
        .padding(16)                                    // 卡片内部留白 16
    }
}
```

带焦点管理的输入框，重点是 `FocusState` 让键盘能正确跳转：

```
struct NameField: View {
    @State private var name = ""
    @FocusState private var isFocused: Bool        // 记录键盘焦点

    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            TextField("物品名称", text: $name)
                .textFieldStyle(.plain)
                .focused($isFocused)               // 绑定到焦点状态
                .submitLabel(.done)                // 键盘右下角显示"完成"
                .onSubmit { isFocused = false }    // 提交后收起键盘
                .padding(.horizontal, 16)          // 页面左右边距 16
                .frame(minHeight: 44)              // 可点区域最小 44
                .background(Color(.secondarySystemGroupedBackground),
                            in: .rect(cornerRadius: 10))   // 输入框圆角 10

            Text("名称会显示在列表和详情页")
                .font(.footnote)                   // 次要信息
                .foregroundStyle(.secondary)
        }
        .padding(16)
    }
}
```

分段选择器，重点是必须给每个选项加 `tag`，否则选中态不会生效：

```
enum SortMode: String, CaseIterable {
    case created = "按创建时间"
    case expire = "按到期时间"
}

struct SortPicker: View {
    @State private var mode: SortMode = .created

    var body: some View {
```

```
        Picker("排序方式", selection: $mode) {
            ForEach(SortMode.allCases, id: \.self) { m in
                Text(m.rawValue).tag(m)            // tag 决定它是哪个选项的值
            }
        }
        .pickerStyle(.segmented)                   // 分段样式，选项少时用
        .padding(16)
    }
}
```

带选择、搜索和滑动操作的列表，这是列表页的标准骨架：

```
struct ItemList: View {
    @State private var items: [Item] = []
    @State private var query = ""
    @State private var selected: Item?

    var body: some View {
        List(selection: $selected) {
            ForEach(items) { item in
                ItemRow(item: item)
                    .swipeActions {
                        Button("删除", role: .destructive) { delete(item) }
                    }
            }
        }
        .searchable(text: $query, prompt: "搜索名称或位置")   // 顶部搜索框
        .refreshable { await reload() }            // 下拉刷新，闭包是 async
    }
}
```

弹出表单页与确认弹窗，重点是 `sheet` 的内容需要自己提供导航栏：

```
struct DetailScreen: View {
    @State private var showEdit = false
    @State private var showDelete = false

    var body: some View {
        ItemDetail()
            .toolbar {
                Button("编辑") { showEdit = true }
            }
            .sheet(isPresented: $showEdit) {
                // sheet 是独立的一层，想要标题栏必须自己包 NavigationStack
                NavigationStack {
                    EditForm()
                        .navigationTitle("编辑物品")
                }
                .presentationDetents([.medium, .large])   // 先半屏，可拖到全屏
            }
            .confirmationDialog("确定删除这个物品吗", isPresented: $showDelete) {
                Button("删除", role: .destructive) { delete() }
                Button("取消", role: .cancel) { }
            }
    }
}
```

读法：`.sheet` 与 `.confirmationDialog` 都挂在触发它的视图上，`isPresented` 绑定一个 `Bool`，系统负责动画和背景遮罩，你只管写内容。危险动作统一用 `role: .destructive`，取消统一用 `role: .cancel`，这样深浅色模式和辅助功能都会自动适配。

控件选择的三条心得

第一，能用系统控件就不要自己拼。系统的 `Button`、`Picker`、`Toggle` 自带点击高亮、VoiceOver 标签、动态字体和深浅色适配，你用 `Text` 加 `onTapGesture` 拼出来的控件这四样全都没有。

第二，外观用 `buttonStyle` 和 `textFieldStyle` 统一改，不要在每个实例上写颜色和边框。改一次主题就能全 App 生效，这是把界面写整齐的关键。

第三，控件的可点区域不要小于 44 x 44。视觉上图标可以是 20 点，用 `.frame(width: 44, height: 44)` 加 `.contentShape(.rect)` 把响应区撑开，用户才按得准。这一条在工具栏图标和列表行内按钮上最容易被忽略。

3.10 反馈与状态提示

一句话结论：任何一个要加载数据的界面，都必须同时准备好加载中、空、失败三种状态，否则用户只会看到一片白。

ProgressView

```
// 不确定进度：只知道在忙，不知道还要多久，用于加载数据
ProgressView("正在加载")

// 确定进度：知道百分比，用于导入、上传这类能算进度的任务
ProgressView(value: 0.4, total: 1.0) {
    Text("正在导入 40%")
}
```

什么时候用：不确定进度放在列表首次加载的位置，确定进度放在能拿到进度的任务上。两者都不要叠在内容上方，应该在内容位置原地替换，这样加载完成时布局不会跳。

ContentUnavailableView 空状态

```
// 标准空状态：一句话说明为什么空，再给一个能推进的动作
ContentUnavailableView {
    Label("还没有物品", systemImage: "shippingbox")
} description: {
    Text("点右上角的加号，记录第一件物品")
} actions: {
    Button("添加物品") { addItem() }
        .buttonStyle(.borderedProminent)
}

// 搜索无结果的固定写法
ContentUnavailableView.search(text: query)
```

什么时候用：列表为空、搜索无结果、功能尚未启用，都用它。它在 iOS 17 起可用，自带插画图标、标题、说明和动作区，比自己拼 `VStack` 规范得多。空状态必须给一条出路，只写数据为空等于把用户堵死。

alert 的两种写法

```
// alert：必须中断的确认，标题加一到两个按钮
.alert("删除后无法恢复", isPresented: $showDelete) {
    Button("删除", role: .destructive) { delete() }
    Button("取消", role: .cancel) { }
} message: {
    Text("这个物品的所有记录都会被移除")
}

// confirmationDialog：多个选项，或者只是想让用户选一个
.confirmationDialog("选择排序方式", isPresented: $showSort) {
    Button("按创建时间") { mode = .created }
    Button("按到期时间") { mode = .expire }
    Button("取消", role: .cancel) { }
}
```

读法：两者都是 iOS 15 起的写法，用 `isPresented` 绑定布尔值，不接受闭包拼接。区别在选择数量：`alert` 出现在屏幕正中，适合必须确认的破坏性操作；`confirmationDialog` 出现在底部，适合多选项和次要的破坏性操作。按钮文字要写清动作，不要出现确定和取消之外的模糊表达。

自制轻提示

系统没有轻提示组件，用 `overlay` 加一个短暂的浮层来实现：

```
struct ToastOverlay: View {
    @State private var toast: String?

    var body: some View {
        Content()
            .overlay(alignment: .bottom) {
                if let toast {
                    Text(toast)
                        .font(.subheadline)
                        .padding(.horizontal, 16)
                        .padding(.vertical, 10)    // 列表行上下留白 10
                        .background(.regularMaterial, in: .capsule)
                        .padding(.bottom, 24)      // 区块与区块之间 24
                        .transition(.move(edge: .bottom).combined(with: .opacity))
                }
            }
            .animation(.snappy, value: toast)      // 值变化时自动播放入场动画
    }
}
```

什么时候用：保存成功、已复制、已收藏这类不需要用户回应的结果。要点是自动消失，通常一到两秒，不要让它常驻，也不要连续弹多个互相遮挡。

加载中、空、失败三种状态

状态	界面表现	必须提供的东西
加载中	原地显示 ProgressView	保持布局高度，避免跳动
空	ContentUnavailableView 加一个动作	说明为什么空和一个出路
失败	说明原因加重新加载按钮	不要只显示错误码

```
enum LoadState {
    case loading
    case empty
    case failed(String)
    case loaded([Item])
}

// 用 switch 一次性写清四种状态，界面里不再出现空白页
switch state {
case .loading:
    ProgressView("正在加载").frame(maxWidth: .infinity, maxHeight: .infinity)
case .empty:
    ContentUnavailableView("还没有物品", systemImage: "shippingbox")
case .failed(let message):
    ContentUnavailableView {
        Label("加载失败", systemImage: "exclamationmark.triangle")
    } description: {
        Text(message)
    } actions: {
        Button("重新加载") { Task { await load() } }
    }
case .loaded(let items):
    ItemList(items: items)
}
```

读法：把状态定义成枚举，让编译器逼着你处理每一种情况，这是避免漏掉空状态最省事的办法。失败状态必须给用户一个能重试的按钮，只显示一条错误信息等于让用户自己刷新。

自制轻提示的时长控制在一个到两秒，出现和消失都用同一个动画，并且同一时刻只允许存在一条。如果短时间内可能连续触发（比如连续收藏多个物品），要么后一条覆盖前一条，要么改成行内状态变化，不要排队弹出一串。

还有一点要提醒：轻提示适合表达已经成功的结果，不适合承载需要用户阅读的信息。需要用户看清楚、或者需要用户决定的内容，用 `alert` 或者页面内的状态说明，不要用一闪而过的浮层。

3.11 动效入门

一句话结论：动画只用来解释变化去了哪里，不用来表演，操作类动画控制在 0.3 秒以内。

withAnimation 与 .animation

```
// 用法一：包住触发状态变化的那行代码，变化发生时播放动画
Button("展开") {
    withAnimation(.snappy) {
        isExpanded.toggle()
    }
}

// 用法二：挂在视图上，指定值变化时自动播放
Text("\(count) 件物品")
    .animation(.snappy, value: count)
```

读法：`withAnimation` 是命令式的，作用于它闭包内的所有状态变化；`.animation(_:value:)` 是声明式的，只有 `value` 变了才播动画，其他变化不受影响。两者选一个就行，同时写会互相覆盖。现在推荐用 `.snappy`，它比旧的 `.easeInOut` 更有质感，也比 `.spring` 更容易预测。

不再使用的写法是 `.animation(.default)` 只带一个参数，它会给所有变化加动画，包括你不想动的那些，已经过时，不要用。

transition 与 matchedGeometryEffect

```
// 入场与出场可以分别指定，动画类型要一致才自然
if isVisible {
    DetailPanel()
        .transition(.move(edge: .bottom).combined(with: .opacity))
}

// matchedGeometryEffect 一句话介绍：让同一个元素在两个位置之间平滑飞行，
// 用于网格小图放大成详情大图这类"同一个东西换了地方"的场景，它需要 @Namespace 配合。
```

`.transition` 只在视图被插入或移除时生效，必须和动画配合，否则会瞬间出现。

`matchedGeometryEffect` 属于进阶能力，这里只需要知道它解决的是位置连续性，普通的状态切换用不上它。

什么时候不该加动画

- 不要给列表滚动加动画，滚动本身有系统惯性，任何附加动画都会造成跟手延迟。
- 不要让动画拖慢操作，弹窗、页面切换、按钮反馈都应在 0.3 秒内结束，超过就会感觉卡。
- 不要给首屏内容加载加入场动画，用户打开 App 的第一目标是尽快看到内容。
- 不要给背景色、文字颜色的变化加长动画，颜色渐变最容易显出延迟。
- 不要为了动画而动画，如果一个动画不能让用户更清楚地知道发生了什么，就删掉它。

给状态切换加一个短动画，这是最值得学的一处：

```
struct FavoriteButton: View {
    @State private var isFavorite = false
```

```
    var body: some View {
        Button {
            withAnimation(.snappy) {                // 只包住状态变化
                isFavorite.toggle()
            }
        } label: {
            Image(systemName: isFavorite ? "star.fill" : "star")
                .foregroundStyle(isFavorite ? .orange : .secondary)   // 收藏用强调色
                .frame(width: 44, height: 44)       // 可点区域最小 44
                .contentShape(.rect)
        }
        .buttonStyle(.plain)
    }
}
```

读法：图标在实心和空心之间切换，颜色跟着状态变化，整个过程 0.2 秒左右。这个动画的作用是告诉用户点击生效了，属于有意义的动画。

给展开收起加动画时，注意同时指定动画和过渡：

```
VStack(alignment: .leading, spacing: 12) {
    Button("展开备注") {
        withAnimation(.snappy) { isExpanded.toggle() }
    }
    if isExpanded {
        Text(item.note ?? "暂无备注")
            .font(.footnote)
            .transition(.opacity.combined(with: .move(edge: .top)))
    }
}
.padding(16)
```

时长对照

场景	建议时长与曲线
按钮反馈、图标切换	.snappy，0.2 秒左右
展开收起一段内容	.snappy，0.25 到 0.3 秒
弹出页面、切换标签	用系统默认，不要自己改
列表增删行	.snappy，且只包住数据变化那一行
列表滚动	不加动画

判断一个动画该不该留的标准很简单：如果把它删掉，用户还能不能看懂发生了什么。能看懂就删，看不懂就留，而且尽量做短。

3.12 用 #Preview 高效迭代设计

一句话结论：`#Preview` 是设计阶段最省时间的工具，为同一个视图写多个状态变体，改样式时一次就能看全所有情况。

最基础的写法，宏直接返回一个预览，不需要 `PreviewProvider`（那是旧写法，已过时，不要用）：

```
#Preview {
    ItemCard(item: Item(name: "移动硬盘", quantity: 2))
}
```

给预览起名字，方便在预览列表里区分：

```
#Preview("卡片：有图片") {
    ItemCard(item: Item(name: "移动硬盘"))
}

#Preview("卡片：无图片") {
    ItemCard(item: Item(name: "备用水杯"))
}
```

名字写在宏的参数位置，预览面板会在设备列表旁边列出所有命名预览，切来切去比重新编译快得多。

固定尺寸预览：`.previewLayout` 已过时，不要用，现在用 `frame` 给预览一个明确的画布，或者用 `#Preview(traits:)` 指定尺寸类别。

```
// 用 frame 限制画布，检查小尺寸下会不会挤
#Preview("小屏宽度") {
    ItemCard(item: Item(name: "移动硬盘"))
        .frame(width: 320)
        .padding()
}

// 用 traits 指定横屏，检查横向布局
#Preview("横屏", traits: .landscapeLeft) {
    ItemDetail(item: Item(name: "移动硬盘"))
}
```

各种修饰符可以直接加在预览内容上，用来模拟系统设置：

```
#Preview("深色模式") {
    ItemList()
        .preferredColorScheme(.dark)              // 固定在深色模式下看对比度
}

#Preview("超大动态字体") {
    ItemList()
        .environment(\.dynamicTypeSize, .accessibility3)   // 模拟辅助功能最大字号
}
```

什么时候用这两个：所有涉及颜色和文字的视图，都应该在深色模式下检查一遍，写死的颜色这时会立刻暴露；所有列表和按钮都应该在 `.accessibility3` 下检查一遍，文字撑破布局同样会立刻暴露。

四个变体的完整示例

这是本项目推荐的标准写法，任何主要界面都照这个模板配四个预览：

```
import SwiftUI

struct ItemDetail: View {
    let item: Item

    var body: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 24) {   // 区块与区块之间 24
                ItemThumbnail(imageData: item.imageData)
                    .frame(maxWidth: .infinity)
                    .frame(height: 220)                  // 详情页头图高度 220
                Text(item.name).font(.largeTitle)        // 页面主标题
                Text("数量 \(item.quantity)")
                    .font(.body)
                    .foregroundStyle(.secondary)
            }
            .padding(16)                                 // 页面左右边距 16
        }
        .background(Color(.systemGroupedBackground))     // 页面底色
    }
}

// 变体 1：正常数据，看默认效果
#Preview("正常") {
    NavigationStack {
        ItemDetail(item: Item(name: "移动硬盘", quantity: 2,
                              location: "书桌第二层"))
    }
}

// 变体 2：没有图片和位置，看占位和空值显示
#Preview("信息缺失") {
    NavigationStack {
        ItemDetail(item: Item(name: "备用水杯"))
    }
}

// 变体 3：深色模式，检查对比度与语义色
#Preview("深色模式") {
    NavigationStack {
        ItemDetail(item: Item(name: "移动硬盘", quantity: 2))
    }
    .preferredColorScheme(.dark)
}

// 变体 4：超大动态字体，检查文字是否撑破布局
#Preview("超大字体") {
    NavigationStack {
        ItemDetail(item: Item(name: "移动硬盘", quantity: 2))
    }
    .environment(\.dynamicTypeSize, .accessibility3)
}
```

读法：四个变体覆盖了最容易出问题的四类输入，正常数据、缺失数据、深色模式、大字体。写样式的时候把这四个预览并排开着，改一行代码四张图同时更新，比来回切换模拟器快得多。

> 结论：为同一个视图写多个状态预览，是这一章里性价比最高的习惯。它把界面自查从凭感觉变成了看一眼。

最后补两个使用习惯。一个是预览里的数据用手写的假数据，不要连真实数据库，预览要能在一秒内刷新出来，连了数据库就会拖慢每一次改样式。另一个是预览要和视图放在同一个文件里，改布局时不用切换文件，滚动距离最短，这也是 `#Preview` 宏相对旧写法的实际好处。

预览跑不起来的时候优先检查三件事：假数据构造器里的参数是不是漏了必填项、`NavigationStack` 是不是包住了需要标题栏的内容、`.preferredColorScheme` 是不是加在了预览内容的根上而不是某个子视图上。这三条能解决大部分预览白屏或报错的情况。

第 4 章 物品管理 App 的界面地图

前三章讲的是设计规范和布局武器库，这一章把它们落到具体题目上：题目一的物品管理 App。这一章不贴成品界面，只做三件事：把题目要求翻译成一张界面清单，把数据模型的每个字段分配到具体界面，把界面之间的跳转关系定下来。做完这三件事，第 5 章的十张任务卡才有地方落地。

4.1 界面地图与难度分级

先给整个 App 列清单。难度按"要你自己决定的事情有多少"判断：★☆☆ 的界面基本由系统控件决定外观，你只负责组织内容；★★☆ 的需要你安排信息层级并处理至少一种状态；★★★ 的系统没有现成控件，从数据结构到视觉呈现都得自己设计。

界面	作用	难度	是否必须
物品列表	一眼看清全部物品，是所有操作的入口	★☆☆	必须
物品详情	展示一件物品的完整信息并提供编辑入口	★☆☆	必须
新增与编辑表单	录入和修改数据，是整个 App 的数据入口	★★☆	必须
分类筛选	按分类缩小列表范围	★☆☆	必须
搜索	按名称或分类快速定位一件物品	★★☆	必须
删除与撤销	移除不再需要的物品，并允许反悔一次	★★☆	必须
分类管理	维护分类清单，避免分类名被写乱	★★☆	建议
设置	默认分类、提醒天数、通知开关	★☆☆	建议
空状态与首次引导	没有数据时告诉用户下一步做什么	★☆☆	建议
统计	用图表回答"我有多少、快过期多少"	★★★	可选
分享卡片	把一件物品导出成图片分享出去	★★★	可选

"必须"一列里的六个界面，直接对应题目基础要求的五条：增删改查、包含名称和分类、本地保存、搜索筛选、完整可用的操作流程。"建议"一列是"不做也能跑通，但做了体验更好"的部分。最后两个属于进阶要求，放在最后做，做不完不影响基础分。

> 判断一个界面该不该现在做，只问一句：不做它，题目基础要求的哪一条会落空。答不上来的，一律往后排。

4.2 数据模型与界面字段对照

界面上的每一块内容都应该能追溯到模型里的一个字段。下表的左列是 model.md 里定义的字段，中间两列是它出现的位置和呈现方式，右列说明它为什么必须或可选。

字段	出现在哪些界面	怎么展示	是否必须
name	列表、详情、表单、搜索、分享卡片	列表主文本用 .body，详情标题用 .title3，表单里是 TextField	必须
categoryName	列表、详情、表单、分类筛选、分类管理、统计	列表次行用 .subheadline，筛选按它分组，统计按它聚合	必须
quantity	列表、详情、表单	列表只在大于 1 时显示，详情单独占一行	建议
expireDate	列表、详情、表单、统计、分享卡片	列表角标显示剩余天数，详情显示完整日期，表单用 DatePicker，统计算临期数量	可选
location	详情、表单、搜索	详情里单独一行，表单里是 TextField	可选
note	详情、表单	详情里是多行文本，表单里是 TextEditor	可选
isFavorite	列表、详情	列表行尾星标，详情放工具栏按钮	建议
tags	详情、表单、搜索	横向排列的胶囊标签，搜索时参与匹配	可选
createdAt	列表排序	不直接展示，只作为排序依据	必须（不展示）
updatedAt	详情	详情底部一行"最近修改"，用 .footnote	可选
imageData	列表、详情、表单、分享卡片	列表缩略图 56 x 56 圆角 10，详情头图高 220 圆角 16	建议
expiryState（派生）	列表、详情、统计	用状态色表达：正常绿色、临期橙色、过期红色	必须（不存库）

必须与可选的界线不看字段本身重不重要，而看"缺了它主流程会不会断"。name、categoryName、createdAt 缺一，列表和排序都无从谈起。quantity、isFavorite、imageData 属于题面里有影子但不致命的部分：题面要求至少包含名称、分类并支持图片，图片之所以归到建议，是因为没有图片的列表依然可用。

再说 expireDate 为什么是可选的。第一层原因是题面：基础要求只写了添加、查看、编辑、删除，时间提醒属于进阶要求的第一条，不做不扣基础分。第二层原因更实际：它是模型里第一个可空字段，一旦允许为空，所有展示它的界面都必须回答"没有日期时显示什么"——列表不能显示"还有 0

天"，详情不能留空白行，统计不能把它算成已过期。这些分支才是真正的成本，所以先跑通主流程，再打开时间显示。

第三，派生状态不要存进数据库。expiryState 每天都会变，存下来就要写定时任务去刷新；用计算属性算，每次读取都拿到当下的值，模型里也不会出现自相矛盾的数据。

> 排序统一用 createdAt 或 updatedAt，不要按 name 排序：中文的字典序结果不符合直觉，用户会以为排序坏了。

4.3 导航结构设计

整个 App 的导航结构如下。方括号里是导航容器，箭头表示跳转方式。

```
RootView（TabView）
|
+-- 页签一：物品
|   +-- NavigationStack
|       +-- ItemListView（列表）
|           +-- push --> ItemDetailView（详情）
|           |              +-- sheet --> ItemFormView（编辑）
|           +-- sheet --> ItemFormView（新增）
|           +-- searchable --> 搜索框
|           +-- alert --> 删除确认
|
+-- 页签二：统计
|   +-- NavigationStack
|       +-- StatsView（统计）
|
+-- 页签三：设置
    +-- NavigationStack
        +-- SettingsView（设置）
            +-- push --> CategoryManagerView（分类管理）
```

为什么两种容器叠在一起用。TabView 负责横向切换"信息域"，NavigationStack 负责纵向进入"更具体的内容"。只用 TabView 的话，详情页也要占一个页签；只用 NavigationStack 的话，统计和设置就得从列表页一路 push 进去。

为什么用三个页签而不是四个。页签的语义是常驻的信息域，新增物品却是一个动作，做成页签就会出现一个点完还停在原地的页签。搜索和分类筛选也不是独立的域，它们只是列表页缩小范围的两个手段，放在列表页内部比放进页签更符合直觉。三个页签刚好覆盖"看数据、看结论、调设置"，超过 5 个时 iPhone 会把多余的折进"更多"，那等于把最难被发现的功能藏起来了。

为什么新增和编辑用 sheet 而不是新页面。push 的含义是"进入下一层"，用户会预期返回时原内容还在。编辑是一件有明确开始和结束的临时任务，sheet 的关闭手势、取消和保存按钮都能自然表

达"这件事结束了"。更关键的是复用：同一个 ItemFormView，传入 nil 是新增，传入已有对象是编辑，只写一套表单。做成 push 的话，新增从列表进入、编辑从详情进入，两条返回路径会让保存逻辑分叉。

为什么删除确认用 alert 而不是 sheet。alert 是中断式确认，职责是"在你继续之前先回答一个是非问题"，点一下就有结果；sheet 是内容容器，寿命更长，里面应该是一段可以操作的任务。删除只有两个选择，塞进 sheet 会多出一次滑动和一次误触的机会。另外 alert 的按钮可以带 role: .destructive，系统自动把文字标红，这比自己画一个红按钮更可靠。需要用户在多个动作里选一个时才改用 confirmationDialog。

列表和详情之间的连接写法固定为"链接给值、目标接收值"两句：

```
NavigationStack {
    List(items) { item in
        NavigationLink(value: item) { ItemRow(item: item) }
    }
    .navigationDestination(for: Item.self) { item in
        ItemDetailView(item: item)
    }
}
```

不要把详情视图直接写进 NavigationLink 的 label 位置，那样列表数据一变化就容易拿到过期的对象。分类管理从设置页 push 进去，因为它是一个有明确层级的次级页面：用户知道自己在设置里面。

4.4 先做哪个界面：推荐顺序

顺序的原则是：先做能验证数据通路的界面，再做需要打磨的界面。

1. 物品列表页。它是整个 App 的骨架：数据从哪来、按什么排序、一行里放什么，这些问题在列表上最先暴露。列表一旦跑起来，之后每写一个界面都有地方看效果。
2. 物品详情页。列表行和详情页共用同一批字段，做完详情你会马上发现列表行里哪些信息是多余的，可以立刻回头精简。
3. 新增与编辑表单。表单是数据的入口，没有它列表永远是空的。
4. 删除与撤销。删除手势长在列表行上，趁列表结构还熟悉的时候顺手做完，成本最低；拖到最后做，你会重新读一遍半个月前写的行视图。

5. 搜索与筛选。数据量小的时候搜索看不出价值，手动录入二三十条之后再写，才测得出匹配和排序的边界情况。
6. 分类管理与设置。这两个界面决定的是分类清单和默认值，属于配置层。等功能稳定了再抽出来做，能避免分类规则反复修改带来的返工。
7. 空状态与首次引导。它必须在"还没有数据"的状态下才能看到效果，等主流程完成后写一个返回空数组的 #Preview 就能验证。
8. 统计页与分享卡片。两者都属于进阶要求：统计要有足够多的数据才有意义，分享卡片依赖前面几章的布局能力，没做完也不影响基础流程。

> 每完成一步都让 App 能编译能跑。不要一次写完五个界面再按编译键：SwiftUI 的报错经常来自你几天前写的那一行，越晚发现越难定位。

第 5 章 十个界面设计任务卡

这一章是整本手册的主体。十张任务卡按第 4 章的界面清单展开，每张卡给目标、给边界、给一份能跑的脚手架，然后把最需要动脑的部分留给你。读完之后你手上应该有一个真正属于自己设计的物品管理 App，而不是一本抄下来的代码。

5.1 怎么用任务卡

任务卡不是教程，是练习单。卡片里的"起步代码"可以整段抄进项目，但里面的留白必须由你填。固定走这四步，顺序不要换。

1. 先读需求。把"界面目标"和"必须出现的信息"读两遍，确认自己知道这个界面存在的理由、以及哪几条信息缺了用户就做不成事。
2. 在纸上画线框。先不打开 Xcode，用笔画出从上到下的结构，标出每一块的高度、间距和字号。画不出来说明信息主次还没想清楚，回到第一步。
3. 再动手写代码。抄起步代码，填留白，随手用 #Preview 看效果，不要等整个项目编译完才看第一眼。
4. 最后对照验收标准自查。每一条标准都要能用眼睛验证：跑一遍、切一次深色模式、删光数据、把每个可点区域都点一次。

每张卡都用八个固定标签分段，标签的顺序就是思考顺序。

标签	它回答的问题
界面目标	这个界面为谁解决什么问题
必须出现的信息	缺了哪一条用户就做不成事
推荐布局	从上到下怎么摆，取值从哪里来
可点区域与交互	哪里能点、点了发生什么
可选创意做法	想加分可以往哪些方向做
验收标准	做到什么程度算完成
起步代码	可以编译运行的脚手架
独立设计提示	哪些地方必须由你决定

> 起步代码只是脚手架。它保证你能编译、能看到一个能跑的空壳，视觉呈现、间距细节、空状态、排序方式全部用注释标了出来。照抄骨架等于什么都没做，留白的地方必须你自己决定。

另外，卡片的编号顺序不等于开发顺序，真正该按什么顺序做，看 4.4 节。

5.2 任务卡一：物品列表页

界面目标：让用户在打开 App 的三秒内看到自己有哪些物品，并一眼认出哪些快到期了。列表不负责展示全部信息，只负责让用户决定点哪一行。

必须出现的信息：

1. 物品名称（name），单行显示，过长时截断而不是换行。

2. 分类名（categoryName），作为第二行的主要信息。

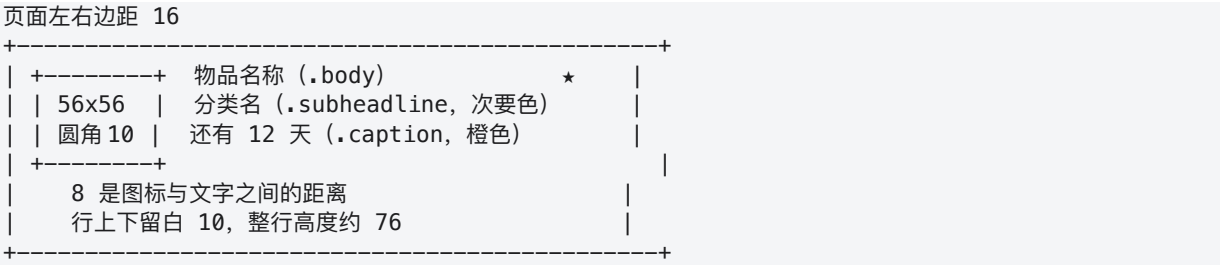
3. 数量（quantity），只在大于 1 时出现，写成"数量 3"而不是一个孤零零的数字。

4. 到期状态，由 expireDate 算出的剩余天数或"已过期"，只在临期和过期时出现。

5. 收藏标记（isFavorite），收藏过的行要有可见星标。

6. 缩略图（imageData），没有图片时用系统符号占位，不要留一块空白。

推荐布局：整页用 List。缩略图 56 x 56、圆角 10、裁剪填充，放在最左，与文字之间留 8；文字竖排两行，名称用 .body，分类用 .subheadline 配次要色，行尾放星标或角标，角标用 .caption。行上下各留 10，页面左右边距 16，一行高度约 76。底色用系统分组背景，靠系统分隔线分层，不要给行加阴影。



可点区域与交互：整行可点，进入详情页，点击区域就是整行 76 的高度，远大于 44 的下限。行尾的收藏星标必须单独可点，点击只切换 isFavorite、不进入详情，所以它要加 .frame(minWidth: 44, minHeight: 44) 和 .contentShape(Rectangle())，否则 20 点的图标很难点中。左滑露出收藏与删除两个动作，做法见任务卡九。列表不需要下拉刷新，@Query 会在数据变化时自动更新。

可选创意做法：

- 左滑收藏，把最常用的动作放进手势，不占行内空间。
- 临期标色：30 天以内用橙色，已过期用红色，正常状态不上色。

验收标准：

- 有数据时每一行都能看到名称和分类，没有任何一行是空白的。
- 没有图片的物品显示占位图标，不会出现一块纯色空白。
- 名称很长时被截断成一行，行高不会突然变高。
- 点击行尾星标能切换收藏状态，同时不会跳进详情页。
- 切换深色模式后，占位底色、主文本、次要文本都清晰可读。

起步代码：

```
import SwiftUI
import SwiftData

struct ItemListView: View {
    @Query(sort: \Item.createdAt, order: .reverse) private var items: [Item]

    var body: some View {
        List {
            ForEach(items) { item in
                ItemRow(item: item)
            }
        }
        .listStyle(.plain)
        .navigationTitle("我的物品")
    }
}

struct ItemRow: View {
    let item: Item

    var body: some View {
        HStack(spacing: 8) {
            // 留给你设计：没有图片时用什么作为缩略图占位
            RoundedRectangle(cornerRadius: 10)
                .fill(Color(.secondarySystemGroupedBackground))
                .frame(width: 56, height: 56)

            VStack(alignment: .leading, spacing: 4) {
                Text(item.name).font(.body)
                // 留给你设计：第二行放分类还是数量，临期用什么状态色
            }
            Spacer()
        }
        .padding(.vertical, 10)
        .padding(.horizontal, 16)
    }
}
```

独立设计提示：

- 列表用单列还是两列网格：以图片为主时网格更好看，以文字和到期时间为主时单列信息密度更高。
- 行里放几行文字、放哪些字段：名称与分类是底线，数量、位置、标签要不要上列表由你决定。

5.3 任务卡二：物品详情页

界面目标：让用户在一屏里确认一件物品的全部信息，并从这里进入编辑，列表放不下的字段都收在这里。

必须出现的信息：

1. 物品名称（name），作为页面主标题。

2. 分类名（categoryName）。

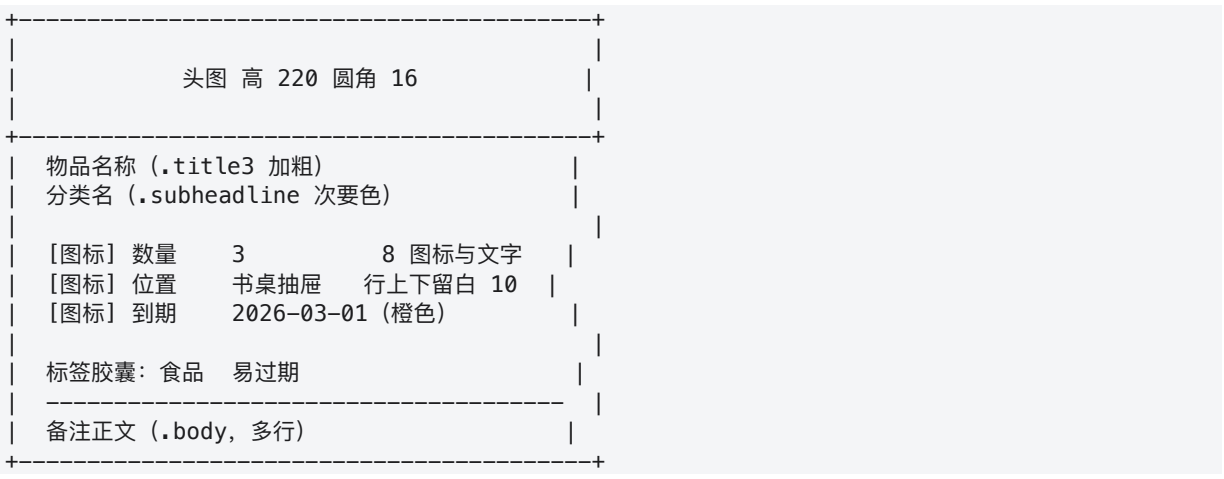
3. 图片（imageData），大图展示，没有图片时给占位。

4. 到期日期（expireDate）与剩余天数，没有日期时整块不出现。

5. 数量（quantity）、存放位置（location）、标签（tags）、备注（note）。

6. 编辑入口、收藏按钮，以及最近修改时间（updatedAt）。

推荐布局：从上到下分四块。头图撑满宽度、高度 220、圆角 16，与下一块之间留 24。标题区名称用 .title3 加粗，分类用 .subheadline 配次要色，两者之间 8。信息区一条一行，图标与文字间距 8，行上下留白 10，标签用 .caption 的胶囊横排。备注用 .body 多行显示。所有内容左右边距 16。整页字号只用 .title3、.body、.subheadline、.caption 四种。



可点区域与交互：头图可点，打开全屏预览或放大；导航栏右侧"编辑"按钮打开表单 sheet，按钮至少 44 x 44；导航栏左侧放收藏按钮，点击切换 isFavorite 并立刻改变颜色；位置那一行可以长按复制。页面整体可滚动。详情页不要放左滑手势，滑动手势属于列表。

可选创意做法：

- 头图加一层渐变遮罩，把名称压在图片上。
- 把到期信息做成一条独立的状态条，用绿、橙、红三色区分。

验收标准：

- 没有图片的物品显示占位图，不会出现一片空白。
- 没有到期日期时，页面上不出现空的日期行，也不会显示"还有 0 天"。
- 备注很长时能滚动看完，不被裁掉。
- 点击编辑能打开表单，且表单里是这件物品的当前数据。
- 切换深色模式后，占位底色与两种文字都对比清晰。

起步代码：

```
import SwiftUI
import SwiftData

struct ItemListWithDetail: View {
    @Query(sort: \Item.updatedAt, order: .reverse) private var items: [Item]

    var body: some View {
        List(items) { item in
            NavigationLink(value: item) {
                Text(item.name).font(.body)
            }
        }
        .navigationDestination(for: Item.self) { item in
            ItemDetailView(item: item)
        }
    }
}

struct ItemDetailView: View {
    @Bindable var item: Item

    var body: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 24) {
                // 留给你设计：头图高 220，没有图片时用什么占位
                RoundedRectangle(cornerRadius: 16)
                    .fill(Color(.secondarySystemGroupedBackground))
                    .frame(height: 220)
                VStack(alignment: .leading, spacing: 8) {
                    Text(item.name).font(.title3.bold())
                    Text(item.categoryName).font(.subheadline).foregroundStyle(.secondary
)
                    // 留给你设计：位置、备注、标签按什么顺序排，放几行
```

```
                }
                .padding(.horizontal, 16)
            }
        }
        .background(Color(.systemGroupedBackground))
        .navigationTitle("物品详情")
    }
}
```

独立设计提示：

- 分几块、怎么合并由你决定：可以把分类与数量并成一行，也可以把到期信息提到最前面。
- 头图是真实照片、系统符号加渐变底色，还是一张色块，由你决定。

5.4 任务卡三：新增与编辑表单

界面目标：让用户在一分钟内录完一件物品，并且清楚哪些字段可以跳过。表单的核心不是好看，而是不容易填错、随时能退出。

必须出现的信息：

1. 名称（name），必填。

2. 分类（categoryName），有默认值"未分类"。

3. 图片选择入口（imageData）。

4. 到期日期（expireDate），默认不启用，打开开关后才出现日期选择器。

5. 数量（quantity）、存放位置（location）、备注（note）。

6. 取消与保存两个出口。

推荐布局：所有字段放进 Form，系统会自动给出分组背景与缩进，不要自己画输入框。依次是基本信息、到期日期、图片三组，数量、位置、备注跟在后面。组内行上下留白按 10 处理，页面左右边距 16，分组标题用 .headline，输入文字用 .body，辅助说明用 .caption。日期开关关闭时 DatePicker 整行不出现，字段不适用时整行隐藏而不是置灰。校验不过时提示写在字段下方，不要弹 alert。

```
+------------------------------------------+
|  取消          新增物品          保存    |
+------------------------------------------+
|  基本信息                                 |
|    名称        [ 输入框              ]    |
|    分类        [ 未分类             > ]   |
+------------------------------------------+
|  到期日期                                 |
|    记录到期日期              [ 开关 ]     |
|    到期日期   2026-03-01    [ 日期 ]      |
+------------------------------------------+
|  图片                                     |
|    选择图片                               |
+------------------------------------------+
```

可点区域与交互：取消和保存都在导航栏，点击区域由系统保证不小于 44 x 44；名称输入框点一下进入编辑并弹出键盘；分类那一行点一下展开选项；日期开关点击切换，DatePicker 点击弹出日历；图片入口唤起系统相册。保存前做一次校验：名称去掉空格后为空就禁用保存按钮，用置灰表达，不要用弹窗。表单要能下滑收拾键盘。编辑已有物品时标题显示"编辑物品"，并且字段里是原值。

可选创意做法：

- 名称与已有物品重复时给一行弱提示，而不是阻止保存。
- 把常用分类做成一行横向胶囊，点一下就填上。

验收标准：

- 名称为空时保存按钮不可点。
- 点取消不会留下任何新数据。
- 编辑已有物品时表单里显示的是原来的值。
- 关闭日期开关后保存，详情页不会出现日期行。
- 键盘弹出时，当前输入框和保存按钮都还看得见。

起步代码：

```
import SwiftUI
import PhotosUI
struct ItemFormView: View {
    @Environment(\.dismiss) private var dismiss
    let item: Item?
    @State private var name = ""
    @State private var categoryName = "未分类"
    @State private var hasExpireDate = false
    @State private var expireDate = Date.now
    @State private var pickedPhoto: PhotosPickerItem?

    var body: some View {
        NavigationStack {
            Form {
                Section("基本信息") {
                    TextField("名称", text: $name)
```

```
                    TextField("分类", text: $categoryName)
                }
                Section("到期日期") {
                    Toggle("记录到期日期", isOn: $hasExpireDate)
                    if hasExpireDate {
                        DatePicker("到期日期", selection: $expireDate,
displayedComponents: .date)
                    }
                }
                Section("图片") {
                    PhotosPicker(selection: $pickedPhoto, matching: .images) {
                        Label("选择图片", systemImage: "photo").frame(minWidth: 44,
minHeight: 44)
                    }
                }
                // 留给你设计：分类选项从哪来，是不是从已有分类里选
                // 留给你设计：校验失败的提示写在哪里，保存按钮什么时候可用
            }
            .navigationTitle(item == nil ? "新增物品" : "编辑物品")
            .toolbar {
                Button("取消") { dismiss() }
                Button("保存") { dismiss() }.disabled(name.isEmpty)
            }
        }
    }
}
```

独立设计提示：

- 字段分几组、每组放几个由你决定：可以按"是什么、什么时候到期、在哪里"来分。
- 分类是下拉选择还是自由输入由你决定，后者灵活但容易写乱。

5.5 任务卡四：分类管理

界面目标：让用户知道现在有哪些分类、每个分类下有多少件物品，并能新增分类。分类管理是数据质量的守门人：分类一旦被随手写成"吃的、食品、食物"，筛选功能就废了。

必须出现的信息：

1. 全部分类名（categoryName 去重后的结果）。

2. 每个分类下的物品数量。

3. 新增分类的入口。

4. 空分类的呈现方式（该不该显示一件物品都没有的分类）。

5. 重命名与删除分类的入口。

推荐布局：用 List 呈现，每行左边分类名、右边数量，行上下留白 10，页面左右边距 16。分类名用 .body，数量用 .caption 配次要色，两者靠 Spacer 撑开。不要给每一行画卡片：分类管理是清单，不是商品列表，加卡片只会让行与行的关系变模糊。新增入口放工具栏，弹一个带输入框的 alert。

可点区域与交互：每行可点，进入该分类下的物品列表；左滑出现重命名与删除；工具栏"新增"按钮至少 44 x 44，点击弹出带输入框的 alert；alert 的确定按钮在名称为空时应当忽略这次输入，因为系统 alert 的按钮不支持动态禁用，判断要写在动作里。删除分类时如果该分类下还有物品，要提示用户这些物品会被归到"未分类"。

可选创意做法：

- 给每个分类配一个固定色和系统符号，列表和筛选共用同一套。
- 把"未分类"的数量标红，提醒用户去整理。

验收标准：

- 新增一个分类后立刻出现在列表里。
- 同名分类不会被创建两次。
- 删除分类后，原分类下的物品不会跟着消失。
- 分类数量与列表页筛选出的条数一致。
- 一个分类都没有时显示提示，而不是空白页。

起步代码：

```
import SwiftUI
import SwiftData

struct CategoryManagerView: View {
    @Query private var items: [Item]
    @State private var newName = ""
    @State private var isAdding = false

    private var counts: [(name: String, total: Int)] {
        Dictionary(grouping: items, by: \.categoryName)
            .map { (name: $0.key, total: $0.value.count) }
            .sorted { $0.total > $1.total }
    }

    var body: some View {
        List {
            ForEach(counts, id: \.name) { entry in
                HStack(spacing: 8) {
                    Text(entry.name).font(.body)
                    Spacer()
                    // 留给你设计：数量用什么颜色和字号，要不要分类色块
                    Text("\(entry.total)").font(.caption).foregroundStyle(.secondary)
                }
                .padding(.vertical, 10)
            }
        }
        .navigationTitle("分类管理")
        .toolbar {
            Button("新增") { isAdding = true }.frame(minWidth: 44, minHeight: 44)
        }
        .alert("新分类", isPresented: $isAdding) {
```

```
            TextField("分类名", text: $newName)
            Button("取消", role: .cancel) { newName = "" }
            // 留给你设计：确定键里怎么查重，新分类要不要立刻设为默认
            Button("确定") { newName = "" }
        }
    }
}
```

独立设计提示：

- 分类允不允许改色、改图标由你决定。
- 分类按名称排序、按物品数量排序还是手动排序由你决定。

5.6 任务卡五：搜索与筛选

界面目标：让用户在物品变多之后仍然能在几秒内找到目标。搜索负责"我知道我要找什么"，筛选负责"我想看看某一类"。

必须出现的信息：

1. 搜索输入框，名称与分类都能匹配。
2. 当前生效的筛选条件。
3. 结果数量，或结果为空时的提示。
4. 每条结果的关键区分信息（名称、分类、到期状态）。
5. 清除关键词与清除筛选的入口。

推荐布局：搜索框由 .searchable 交给系统，它会自己出现在导航栏下方，不要自己画一个输入框。筛选控件放在工具栏或列表上方，用分段控件或菜单。结果列表沿用列表页的行布局：缩略图 56 x 56 圆角 10，图标与文字间距 8，行上下留白 10，页面左右边距 16。结果为空时用系统的搜索空状态，文案写明"没有找到"，而不是一片白。

可点区域与交互：点击搜索框进入输入，键盘上带搜索键；点击筛选控件切换条件，切换后列表立刻更新；结果行可点进入详情；搜索框右侧的系统清除按钮点一下清空关键词。搜索页不要再叠加左滑手势以外的自定义手势，那会和列表本身的手势冲突。筛选控件的每个选项都要保证点击区域不小于 44 x 44。

可选创意做法：

- 保留最近几条搜索词，点一下直接复用。

- 把筛选做成一行横向滚动的胶囊标签，比分段控件更省空间。

验收标准：

- 输入两个字就能滤掉大部分行，结果里不会混进不匹配的物品。
- 切换筛选后结果数量随之变化。
- 清空关键词后恢复全部结果。
- 无结果时显示提示和清除入口，而不是空白。
- 从搜索结果进入详情再返回，关键词与筛选条件仍然保留。

起步代码：

```
import SwiftUI
import SwiftData

struct ItemSearchView: View {
    @Query(sort: \Item.expireDate) private var items: [Item]
    @State private var keyword = ""
    @State private var onlyFavorite = false

    private var results: [Item] {
        items.filter { item in
            let hitKeyword = keyword.isEmpty
                || item.name.localizedStandardContains(keyword)
                || item.categoryName.localizedStandardContains(keyword)
            let hitFavorite = !onlyFavorite || item.isFavorite
            // 留给你设计：临期和过期怎么参与筛选，用派生状态判断
            return hitKeyword && hitFavorite
        }
    }

    var body: some View {
        List(results) { item in
            VStack(alignment: .leading, spacing: 4) {
                Text(item.name).font(.body)
                // 留给你设计：搜索结果里还要显示哪些字段才够判断
                Text(item.categoryName).font(.subheadline).foregroundStyle(.secondary)
            }
        }
        .searchable(text: $keyword, prompt: "搜索名称或分类")
        .toolbar {
            Toggle("只看收藏", isOn: $onlyFavorite)
        }
    }
}
```

独立设计提示：

- 搜索匹配哪些字段由你决定：名称和分类是底线，标签、备注、位置要不要一起搜由你决定。
- 筛选用分段控件、下拉菜单还是胶囊标签由你决定。

5.7 任务卡六：空状态与首次引导

界面目标：在没有数据、搜索无结果、筛选过严这三种"什么都没有"的时刻，告诉用户发生了什么、下一步做什么。空状态不是装饰，它承担的是第一次使用的引导。

必须出现的信息：

1. 一句说明现状的标题。

2. 一句解释原因或价值的说明。

3. 一个明确的下一步按钮。

4. 一个与语境匹配的系统符号，不要用警告三角。

5. 首次引导里说明数据存在本机、不会上传。

推荐布局：内容整体居中，顺序是图标、标题、说明、按钮，四者之间只用 8 和 24 两档间距。图标用系统符号，字号取 .largeTitle 的两到三倍；标题用 .headline，说明用 .subheadline 配次要色，按钮用 .borderedProminent 并保证高度 44 以上。页面底色沿用系统分组背景，不要换成大面积色块。首次引导如果用 sheet，右上角必须有跳过。

可点区域与交互：按钮至少 44 x 44，点击后直接打开新增表单；说明文字不可点；如果空状态来自搜索，按钮文案换成"清除筛选"。整块空状态本身不要做成可点的，避免用户误触后不知道发生了什么。首次引导的跳过按钮也要满足 44 x 44，点了之后不能再自动弹出来。

可选创意做法：

- 三种空状态各写一套文案："还没有物品""没有找到""还没有收藏"。
- 首次引导用三页图文，而不是一个弹窗。

验收标准：

- 删掉全部数据后打开 App，看到的是提示和按钮，不是白色空白页。
- 点空状态里的按钮能直接打开新增表单。
- 搜索无结果时的文案与无数据时不同。
- 切换深色模式后插画与文字都清晰。
- 首次引导可以跳过，并且不会被再次自动弹出。

起步代码：

```
import SwiftUI
import SwiftData

struct ItemListView: View {
    @Query private var items: [Item]
    @State private var showingForm = false

    var body: some View {
        Group {
            if items.isEmpty {
                ContentUnavailableView {
                    Label("还没有物品", systemImage: "shippingbox")
                } description: {
                    Text("记下第一件物品，之后就能看到它什么时候到期。")
                } actions: {
                    Button("添加第一件物品") { showingForm = true }
                        .buttonStyle(.borderedProminent)
                        .frame(minWidth: 44, minHeight: 44)
                }
                // 留给你设计：插画用哪个系统符号，文案用什么语气
            } else {
                List(items) { item in
                    Text(item.name).font(.body)
                }
            }
        }
        .sheet(isPresented: $showingForm) {
            // 留给你设计：首次引导是一页图文还是三步说明
            Text("引导页")
        }
    }
}
```

独立设计提示：

- 文案语气由你决定：正式、轻松还是带一点幽默，但要整篇统一。
- 插画用单个系统符号，还是符号加一块浅色底，由你决定。

5.8 任务卡七：统计页

界面目标：回答两个问题——我总共有多少件物品，其中有多少快要到期。统计页不承担操作，只呈现结论。

必须出现的信息：

1. 物品总数。

2. 分类维度的数量分布，用图表表达。

3. 临期数量与已过期数量。

4. 图表的可读标注：每个柱子或扇区代表哪个分类。

5. 没有数据时的提示。

推荐布局：自上而下三块。顶部放两到三个数字卡片，卡片内边距 16、圆角 16、卡片之间 12，数字用 .title3、标签用 .caption 配次要色。中间是图表，高度 220，与上下两块各留 24。页面左右边距 16，底色用系统分组背景，卡片用次级分组背景形成色差，不要加阴影。图表配色只用一个强调色加最多三个状态色，不要给每个分类换颜色。

可点区域与交互：数字卡片可点，跳到筛选好的列表；图表的数据点在 iOS 上可以点选并高亮，如果实现了就要给出选中反馈；页面整体可滚动。统计页不放删除与编辑动作，避免用户在看数据时误操作。

可选创意做法：

- 把"临期占比"做成一个环形进度。
- 用折线看每月新增数量，回答"我最近是不是买多了"。

验收标准：

- 页面上的总数与列表里的行数一致。
- 数据为空时显示提示，而不是一张空白图表。
- 切换深色模式后图表标签仍然清楚可读。
- 窄屏上图表标签不会互相遮挡。
- 没有到期日期的物品不会被算进临期数量。

起步代码：

```
import SwiftUI
import SwiftData
import Charts

struct StatsView: View {
    @Query private var items: [Item]

    private var byCategory: [(name: String, total: Int)] {
        Dictionary(grouping: items, by: \.categoryName)
            .map { (name: $0.key, total: $0.value.count) }
            .sorted { $0.total > $1.total }
    }

    var body: some View {
        NavigationStack {
            ScrollView {
                VStack(alignment: .leading, spacing: 24) {
                    Text("按分类统计").font(.headline)
                    // 留给你设计：用柱状图还是扇区图，配色只用一个强调色
                    Chart(byCategory, id: \.name) { entry in
                        BarMark(
                            x: .value("分类", entry.name),
                            y: .value("数量", entry.total)
```

```
                        )
                    }
                    .frame(height: 220)
                    // 留给你设计：还要放哪些数字，临期数量怎么算出来
                }
                .padding(16)
            }
            .background(Color(.systemGroupedBackground))
            .navigationTitle("统计")
        }
    }
}
```

独立设计提示：

- 图表用柱状图、扇区图还是折线图由你决定，选完要能说清为什么。
- 分类超过六个时怎么处理小项由你决定：合并成"其他"还是横向滚动。

5.9 任务卡八：设置页

界面目标：把默认值和偏好集中到一个地方，让主流程少问几次同样的问题。设置页不做业务，它只影响别的界面的行为。

必须出现的信息：

1. 默认分类。

2. 默认的提前提醒天数。

3. 到期通知开关。

4. 分类管理的入口。

5. 版本信息与一句数据说明（数据保存在本机）。

推荐布局：用 Form 分组呈现，每组一个小标题，组内行高交给系统，组与组之间按 24 处理。设置项左文字右控件：文字用 .body，补充说明用 .caption 放在下一行，不要硬挤在同一行。需要进入下一层的项目用带箭头的行，开关和选择器就地操作。页面左右边距 16，不要给每个设置项配彩色图标底，那是设置页变花的最快方式。

可点区域与交互：带箭头的行整行都可点，不只是箭头那个小区域；开关点一下切换；选择器点击展开列表。通知开关打开时要申请权限，被拒绝后在行下方补一句说明并给出跳转系统设置的入口，不要反复弹窗。所有交互控件的点击区域不小于 44 x 44。

可选创意做法：

- 把提醒设置做成"提前几天"的几个档位，而不是让用户自己填数字。
- 加一个"清除全部数据"的破坏性操作，并且二次确认。

验收标准：

- 改完默认分类后新建物品，带出来的确实是新分类。
- 关闭通知开关后不再出现系统权限弹窗。
- 设置项在深色模式下都清晰可读。
- 分类管理能从设置进入，也能返回。
- 重启 App 后设置值仍然保留。

起步代码：

```
import SwiftUI

struct SettingsView: View {
    @AppStorage("defaultCategory") private var defaultCategory = "未分类"
    @AppStorage("remindDays") private var remindDays = 30
    @AppStorage("notifyEnabled") private var notifyEnabled = false

    private let categories = ["未分类", "食品", "药品", "数码", "证件"]

    var body: some View {
        NavigationStack {
            Form {
                Section("默认设置") {
                    Picker("默认分类", selection: $defaultCategory) {
                        ForEach(categories, id: \.self) { Text($0) }
                    }
                    Picker("提前提醒", selection: $remindDays) {
                        Text("7 天").tag(7)
                        Text("30 天").tag(30)
                    }
                }
                Section("通知") {
                    Toggle("到期提醒", isOn: $notifyEnabled)
                    // 留给你设计：没有通知权限时的说明放在哪里
                }
                Section {
                    Text("版本 1.0").font(.footnote).foregroundStyle(.tertiary)
                    // 留给你设计：还要放哪些入口，例如数据导出和关于
                }
            }
            .navigationTitle("设置")
        }
    }
}
```

独立设计提示：

- 哪些选项值得进设置、哪些该直接放在列表页由你决定。
- 分成几组、顺序怎么排由你决定：可以按"录入习惯、提醒、数据"来分。

5.10 任务卡九：删除与撤销

界面目标：让删除既顺手又不容易后悔，做到一次手势、一次确认、一次反悔。

必须出现的信息：

1. 被删除物品的名称，确认框里必须写清楚删的是哪一件。

2. 删除动作本身，由左滑露出。

3. 二次确认。

4. 撤销入口。

5. 删除后的结果反馈：列表少了一行。

推荐布局：删除动作挂在列表行上，从左滑出红色"删除"按钮，按钮高度铺满行高、至少 44。确认框交给系统的 alert，不要自己画。撤销提示条放在屏幕底部安全区上方，宽度是页面宽减去左右各 16，内部留 12 到 16，圆角用胶囊或 10，文字用 .subheadline，撤销按钮用强调色，停留三到五秒。提示条不能挡住列表最后一行，列表底部要留出空间。

可点区域与交互：左滑露出按钮，关闭全滑直接删除，让用户必须再点一次；点删除弹出确认 alert；alert 里取消按钮默认加粗，删除按钮带 destructive 角色所以系统自动标红；提示条上的"撤销"点一下恢复数据；提示条出现时点别处可以立刻收起。删除只保留一条入口，不要在详情页再放一个删除按钮，否则确认逻辑要维护两套。

可选创意做法：

- 长按进入多选，一次删除多件物品。
- 删除瞬间给一次轻微触觉反馈。

验收标准：

- 左滑能露出删除按钮，再点一次才真正删除。
- 确认框里能看到被删物品的名字。
- 点取消后物品仍然在列表里。
- 删除后出现撤销提示，点撤销能恢复。
- 一次全滑不会绕过确认直接删除。

起步代码：

```
import SwiftUI
import SwiftData

struct ItemListView: View {
    @Environment(\.modelContext) private var context
    @Query private var items: [Item]
    @State private var pending: Item?
    @State private var isConfirming = false
    @State private var undoItem: Item?
    var body: some View {
        List {
            ForEach(items) { item in
                Text(item.name).font(.body)
                    .swipeActions(edge: .trailing, allowsFullSwipe: false) {
                        Button(role: .destructive) {
                            pending = item
                            isConfirming = true
                        } label: {
                            Label("删除", systemImage: "trash")
                        }
                    }
                    // 留给你设计：左滑还放哪些动作，收藏排在第几个
            }
        }
        .alert("删除这件物品？", isPresented: $isConfirming) {
            Button("取消", role: .cancel) { pending = nil }
            Button("删除", role: .destructive) { deletePending() }
        } message: {
            Text("删除后还可以在提示条里撤销一次。")
        }
        // 留给你设计：撤销提示条放底部还是安全区上方，几秒后消失
    }

    private func deletePending() {
        guard let item = pending else { return }
        undoItem = item
        context.delete(item)
        pending = nil
    }
}
```

独立设计提示：

- 撤销停留几秒由你决定，太短来不及反应，太长挡住内容。
- 确认文案的语气由你决定，但要写清楚删的是哪一件。

5.11 任务卡十：分享卡片页

界面目标：把一件物品导出成一张图片分享给别人，对应进阶要求里的"调用系统分享能力"。

必须出现的信息：

1. 卡片上的物品名称与分类。

2. 到期日期或剩余天数，前提是这件物品有日期。

3. 图片，前提是这件物品有图片。

4. 一个代表 App 的标识或一句标语。

5. 生成与分享两个动作。

推荐布局：屏幕上是"预览加操作"两段式。上方是卡片预览，宽 300、圆角 16、内边距 16，卡片内名称用 .title3、分类用 .subheadline、日期用 .caption。下方是"生成分享图"和"分享"两个按钮，两者之间 24，高度都不少于 44，页面整体左右边距 16。预览的视觉必须与导出的图片完全一致；导出时固定用浅色背景，深色模式下分享出去的图才不会变成黑底白字。

可点区域与交互：生成按钮点击后把卡片渲染成图片；分享按钮只在图片生成之后出现，点击唤起系统分享面板；预览卡片长按可以快速保存到相册；两个按钮都要满足 44 x 44。渲染失败时要给一句可见的提示，不要静默失败。分享面板里用户取消分享时，页面状态要保持不变。

可选创意做法：

- 提供方形和竖版两种卡片模板，让用户自己挑。
- 在卡片角落加日期水印或一句标语。

验收标准：

- 生成的图片与屏幕上的预览一致。
- 分享面板能正常唤起，并能完成一次分享。
- 没有图片的物品也能生成一张不空白的卡片。
- 图片还没生成时点分享不会报错或崩溃。
- 深色模式下导出的图片文字仍然清楚。

起步代码：

```
import SwiftUI
import UIKit
struct ShareCardView: View {
    let item: Item
    @State private var cardImage: Image?
    var body: some View {
        VStack(spacing: 24) {
            card
            Button("生成分享图") { render() }
                .frame(minWidth: 44, minHeight: 44)
            if let cardImage {
                ShareLink(item: cardImage, preview: SharePreview(item.name, image:
cardImage)) {
                    Label("分享", systemImage: "square.and.arrow.up")
                        .frame(minWidth: 44, minHeight: 44)
                }
            }
        }
```

```
        .padding(16)
    }

    // 留给你设计：卡片露出哪些字段，背景用纯色还是渐变
    private var card: some View {
        VStack(alignment: .leading, spacing: 8) {
            Text(item.name).font(.title3.bold())
            Text(item.categoryName).font(.subheadline).foregroundStyle(.secondary)
        }
        .padding(16)
        .frame(width: 300, height: 400, alignment: .topLeading)
        .background(Color(.secondarySystemGroupedBackground), in:
RoundedRectangle(cornerRadius: 16))
    }

    @MainActor private func render() {
        let renderer = ImageRenderer(content: card)
        renderer.scale = 3
        // 留给你设计：导出尺寸多大，要不要加水印或 App 名称
        if let uiImage = renderer.uiImage {
            cardImage = Image(uiImage: uiImage)
        }
    }
}
```

独立设计提示：

- 卡片是方形、竖版还是横版由你决定，但导出后要能在聊天窗口里看清。
- 背景用纯色、渐变还是物品自己的图片由你决定。

第 6 章 自己造可复用的组件

前一章你已经能照着抄出能跑的界面，但抄出来的东西往往是一次性的：同一张卡片在列表页、搜索页、收藏页各写一遍，改一个圆角要改三个文件。这一章把界面里反复出现的部分提炼成组件（Component），并让它经得起改。

6.1 什么时候该把界面拆成组件

拆组件不是"拆得越多越专业"。动手之前用下面三条标准判断，任意一条成立就值得拆。

- 同一个东西出现三次以上。同一种卡片、同一种角标、同一种空状态视图，写到第三遍时复制粘贴的代价已经超过抽象的成本。
- 需要单独预览调试。为了看清一个角标的样子，每次都要启动模拟器、点进列表、造一条临期数据，说明这个部分需要一个自己的 `#Preview`。
- 有独立的状态。一个小组件自己带 `@State`（比如卡片上的展开收起），留在父视图里会让父视图的状态越来越多、越来越难读。

三条都不成立就别拆——这叫过早抽象（Premature Abstraction）。它的代价很具体：多出一个类型、多一层参数传递，参数一旦不够用就要回来改所有调用点；而抽象错了比没有抽象更难改，因为你已经有三处在依赖它。一个只在一个地方出现、不会有第二种用法的 `SectionHeader`，就写在那一个文件里。

> 结论：拆组件的收益来自"重复"和"独立预览"，不来自"文件多"。先用三条标准筛一遍，通过再拆。

6.2 从硬编码到可配置：一个物品卡片的演进

这一节用同一个"物品卡片"走三步：先写死在列表里，再抽成结构体接收数据，最后让它既能用于列表也能用于网格。字段名与统一数据模型一致：名称 `name`、分类 `categoryName`、到期日期 `expireDate`。

第一步：写死在列表里。这是绝大多数人第一次写出来的东西——一个临期角标、一张 56 x 56 圆角 10 的缩略图、两行文字，全部堆在列表的行内。

```
import SwiftUI

struct ItemListInlineView: View {
```

```
    let items: [Item]

    var body: some View {
        List(items) { item in
            HStack(spacing: 12) {
                thumbnail(for: item)
                VStack(alignment: .leading, spacing: 4) {
                    Text(item.name).font(.body)
                    Text(item.categoryName).font(.footnote).foregroundStyle(.secondary)
                }
                Spacer(minLength: 0)
                if let date = item.expireDate, date < .now.addingTimeInterval(2592000) {
                    Text("临期")
                        .font(.caption2)
                        .foregroundStyle(.orange)
                        .padding(.horizontal, 8).padding(.vertical, 4)
                        .background(.orange.opacity(0.12), in: Capsule())
                }
            }
            .padding(.vertical, 10)
        }
    }

    @ViewBuilder
    private func thumbnail(for item: Item) -> some View {
        if let data = item.imageData, let image = UIImage(data: data) {
            Image(uiImage: image).resizable().scaledToFill()
                .frame(width: 56, height: 56)
                .clipShape(RoundedRectangle(cornerRadius: 10))
        } else {
            RoundedRectangle(cornerRadius: 10)
                .fill(Color(.secondarySystemFill))
                .frame(width: 56, height: 56)
        }
    }
}
```

这段能跑，问题要到复用时才暴露：搜索结果页、收藏页再画一遍，上面这三十多行就被复制第二、第三遍；临期判定条件、圆角 10、间距 12 散落在每份副本里，改一次要改全部。

第二步：抽成 `ItemCard`，通过参数接收数据。关键决定是"组件接收什么"——只传它真正要画的字段，而不是把整个 `Item` 传进去。

```
struct ItemCard: View {
    let name: String
    let categoryName: String
    let expireDate: Date?
    let image: Image?

    var body: some View {
        HStack(spacing: 12) {
            thumbnail
            VStack(alignment: .leading, spacing: 4) {
                Text(name).font(.body).lineLimit(2)
                Text(categoryName).font(.footnote).foregroundStyle(.secondary)
            }
            Spacer(minLength: 0)
            if let expireDate { ExpiryBadge(date: expireDate) }
        }
        .padding(12)
        .background(Color(.secondarySystemGroupedBackground),
```

```
                    in: RoundedRectangle(cornerRadius: 16))
    }

    @ViewBuilder
    private var thumbnail: some View {
        if let image {
            image.resizable().scaledToFill()
                .frame(width: 56, height: 56)
                .clipShape(RoundedRectangle(cornerRadius: 10))
        } else {
            RoundedRectangle(cornerRadius: 10)
                .fill(Color(.secondarySystemFill))
                .frame(width: 56, height: 56)
        }
    }
}

struct ExpiryBadge: View {
    let date: Date

    private var isExpired: Bool { date < .now }
    private var isSoon: Bool { !isExpired && date < .now.addingTimeInterval(2592000) }

    var body: some View {
        if isExpired || isSoon {
            Text(isExpired ? "已过期" : "临期")
                .font(.caption2)
                .foregroundStyle(isExpired ? .red : .orange)
                .padding(.horizontal, 8).padding(.vertical, 4)
                .background((isExpired ? Color.red : Color.orange).opacity(0.12),
                            in: Capsule())
        }
    }
}
```

为什么这样改：`ItemCard` 只依赖四个参数，可以脱离 SwiftData 独立预览；`ExpiryBadge` 把两种状态收进一个类型，颜色和判定只有一处；卡片内留白统一 12、圆角统一 16，数值都来自设计规范，不再散落在调用点。

第三步：把右侧的附加内容做成 `@ViewBuilder` 参数。列表右侧放角标、网格右侧放收藏按钮，如果继续加 `showsBadge` 这类开关，参数会越来越多。正确做法是让调用方把这一段塞进来。

```
struct ItemCard<Trailing: View>: View {
    let name: String
    let categoryName: String
    let image: Image?
    @ViewBuilder let trailing: Trailing

    init(name: String, categoryName: String, image: Image? = nil,
         @ViewBuilder trailing: () -> Trailing) {
        self.name = name
        self.categoryName = categoryName
        self.image = image
        self.trailing = trailing()
    }

    var body: some View {
        HStack(spacing: 12) {
            thumbnail
            VStack(alignment: .leading, spacing: 4) {
```

```
                Text(name).font(.body).lineLimit(2)
                Text(categoryName).font(.footnote).foregroundStyle(.secondary)
            }
            Spacer(minLength: 0)
            trailing
        }
        .padding(12)
        .background(Color(.secondarySystemGroupedBackground),
                    in: RoundedRectangle(cornerRadius: 16))
    }

    @ViewBuilder
    private var thumbnail: some View {
        if let image {
            image.resizable().scaledToFill()
                .frame(width: 56, height: 56)
                .clipShape(RoundedRectangle(cornerRadius: 10))
        } else {
            RoundedRectangle(cornerRadius: 10)
                .fill(Color(.secondarySystemFill))
                .frame(width: 56, height: 56)
        }
    }
}
```

`@ViewBuilder` 修饰的闭包返回视图，SwiftUI 会把闭包里的多个视图自动合成一个。它有显式泛型（如上面的 `Trailing`）和 `some View` 两种形态：泛型版能接收任意视图，但不能把结果放进 `[ItemCard]` 数组混装；需要混装时改用 `some View` 的写法。

同一个 `ItemCard` 现在既能进列表也能进网格，调用方只在尾部换一个视图。

```
struct ItemListScreen: View {
    let items: [Item]

    private let columns = [GridItem(.adaptive(minimum: 150), spacing: 12)]

    private func thumbnailImage(for item: Item) -> Image? {
        guard let data = item.imageData, let ui = UIImage(data: data) else { return nil }
        return Image(uiImage: ui)
    }

    var body: some View {
        List(items) { item in
            ItemCard(name: item.name,
                     categoryName: item.categoryName,
                     image: thumbnailImage(for: item)) {
                ExpiryBadge(date: item.expireDate ?? .distantFuture)
            }
        }
        .listStyle(.plain)
    }
}

struct ItemGridScreen: View {
    let items: [Item]

    private let columns = [GridItem(.adaptive(minimum: 150), spacing: 12)]

    var body: some View {
        ScrollView {
            LazyVGrid(columns: columns, spacing: 12) {
```

```
                ForEach(items) { item in
                    ItemCard(name: item.name, categoryName: item.categoryName) {
                        if item.isFavorite {
                            Image(systemName: "star.fill").foregroundStyle(.orange)
                        }
                    }
                }
            }
            .padding(16)
        }
        .background(Color(.systemGroupedBackground))
    }
}
```

三步下来外观没变，但数据来源、附加内容和尺寸数值全部从写死变成了可配置。以后要做横版卡片，只需加一个布局参数，而不是再复制四十行。

6.3 自定义 ViewModifier 与 ButtonStyle

`ItemCard` 解决"一块界面重复"，`ViewModifier` 解决"一串修饰符重复"。背景加圆角加内边距的组合在五个地方出现时，把它封成一个修饰符。

```
struct CardAppearance: ViewModifier {
    var padding: CGFloat = 12
    var cornerRadius: CGFloat = 16

    func body(content: Content) -> some View {
        content
            .padding(padding)
            .background(Color(.secondarySystemGroupedBackground),
                        in: RoundedRectangle(cornerRadius: cornerRadius))
    }
}

extension View {
    func cardAppearance(padding: CGFloat = 12,
                        cornerRadius: CGFloat = 16) -> some View {
        modifier(CardAppearance(padding: padding, cornerRadius: cornerRadius))
    }
}
```

用法就是一句 `.cardAppearance()`，需要时覆盖默认值：`.cardAppearance(padding: 16)`。选择哪条路有一条规则：只给 `View` 加一层固定样式，写成 `extension View` 加普通方法；需要拿到 `Content` 才能决定怎么包裹，或者需要被组合复用，就写 `ViewModifier`。后者的 `body(content:)` 拿到的是被修饰的视图本身，只有它能做"前后各包一层"。

`ButtonStyle` 管按钮的外观与按下反馈，必须实现 `makeBody(configuration:)`，参数里的 `configuration` 提供 `label` 和 `isPressed`。

```
struct PrimaryButtonStyle: ButtonStyle {
    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.body)
            .foregroundStyle(.white)
            .padding(.horizontal, 16)
            .frame(minHeight: 44)
            .background(Color.accentColor.opacity(configuration.isPressed ? 0.7 : 1),
                        in: RoundedRectangle(cornerRadius: 10))
    }
}

Button("添加物品") { addItem() }
    .buttonStyle(PrimaryButtonStyle())
```

`configuration.isPressed` 是 `.buttonStyle` 相对自己加背景的唯一优势：按下状态由系统管理，你不用手写手势。

> 提示：`ViewModifier` 的 `body` 只写样式，不写业务逻辑。它一旦开始读状态、发请求，就已经是视图而不是修饰符了。

6.4 组件 API 的设计原则

组件是给别人（包括三个月后的你）用的，参数设计比内部实现更重要。

- 参数名用领域词，不用通用词。`expireDate`、`categoryName` 比 `date`、`subtitle` 好，因为调用点只有领域词能自我解释。
- 布尔参数用断言式命名。写 `showsBadge`、`isCompact`，不要写 `badge`、`compact`，让调用处能读成一句话。
- 参数不超过 5 个。到第 6 个时先停下：通常说明这几项该合成一个配置结构体，或者这个组件承担了两件事，应该拆开。
- 默认值只给"九成场景都一样"的参数。内边距、圆角适合给默认值；`name`、`categoryName` 这类语义内容绝不能给默认值，否则某天会有一张空白卡片悄悄出现。
- 优先传值，只在组件需要改数据时才传绑定。纯展示组件收 `let name: String`；输入框、滑块这类需要写回时才收 `@Binding`。
- 组件不自己管业务状态。`ItemCard` 不该持有 `@Query`，也不该发网络请求，它只画你给它的东西；它可以有展开、高亮这类视觉状态。

文件层面的约定如下。

项目	建议	原因
文件行数	单个可复用组件 150 行以内	超过说明里面塞了多个组件
一个文件	一个主要组件加它的私有子视图	便于在导航栏一眼找到类型
预览数量	每个组件至少 2 个 `#Preview`	分别覆盖正常数据与超长名称这类极端数据
命名	用名词结尾，如 `ItemCard`	`ItemCardView` 这种重复后缀可以省掉

> 结论：好组件的标准不是"功能多"，而是"调用点短"。调用时要填七八个参数、还要先看实现才知道哪个必填，就说明它还没封装好。

第 7 章 设计自查清单

这一章是一份可以逐条打勾的清单。提交界面之前从 7.1 走到底，每一条都在模拟器里点一遍。凡是说不清怎么验证的条目都已经删掉了，剩下的每一条都能在 App 里直接看到结果。

7.1 布局与间距

- 页面左右边距是否都是 16 点，滚到底部时最后一个元素的下边距是否一致。
- 所有 `padding` 与 `spacing` 是否都是 4 的倍数，有没有 7、13 这类随手写的数值。
- 卡片内部留白是否只用 12 或 16，同一屏里的卡片是否用同一个数值。
- 列表行的上下留白是否是 10，行与行之间有没有空行或压在一起。
- 卡片与卡片之间是否是 12，区块与区块之间是否是 24，两者有没有用混。
- 一个界面的圆角是否不超过两种，缩略图 10、卡片 16 是否与规范一致。
- 文字与卡片边缘之间是否有留白，有没有文字贴边或被圆角切掉。
- 可点元素的点击区域是否达到 44 x 44，20 点的图标是否补了 `contentShape` 或最小尺寸。
- 换成最小屏幕机型后，横向排列的元素有没有被挤出屏幕或用省略号硬顶。
- 长内容出现时该换行的文字是否换行，该省略的是否有 `lineLimit`，有没有溢出被裁掉。
- 同一行里多个元素的基线是否对齐，图标和文字有没有上下错半个字。

7.2 内容与状态完整性

- 加载中是否显示了进度提示，而不是一片空白。
- 空数据时是否显示提示文案，文案里是否说明了下一步该做什么。
- 空数据页面是否提供直接操作的按钮，而不是让用户自己找入口。
- 加载失败时是否显示错误说明和"重试"按钮，重试能否真的重新发起请求。
- 断网时触发加载是否给出可理解的提示，App 有没有卡住或崩溃。
- 数据很多时（例如 500 条）列表是否流畅滚动，是否用了 `LazyVStack` 或 `LazyVGrid`。
- 数据很多时顶部是否有搜索或筛选入口，用户不必手动翻到底。
- 超长名称（例如 50 个汉字）是否有 `lineLimit`，会不会挤掉同行的其他信息。
- 长备注、长地址是否有展开或查看详情入口，而不是被截断且无法查看。
- 没有图片时是否显示占位图形，占位图形与有图尺寸是否一致，列表有没有因此跳动。

- 数量、分类、位置等可选字段为空时，是否隐藏该项而不是显示"nil"或空白行。
- 每个可点元素点击后是否有可见变化，用户能否判断自己点到了哪里。

7.3 视觉与配色

- 主文字是否用 `.primary`，次要文字是否用 `.secondary`，更弱的说明是否用 `.tertiary`。
- 有没有写 `Color.white` 当背景、`Color.black` 当文字，这类写法在深色模式下会失效。
- 页面底色是否用 `Color(.systemGroupedBackground)`，卡片底色是否用 `Color(.secondarySystemGroupedBackground)`。
- 强调色是否只在根视图写了一处 `.tint`，有没有在子视图里零散写 `Color.blue`。
- 状态色是否只表达状态（正常绿、临期橙、过期红），有没有拿它做装饰。
- 一个界面的颜色是否控制在 1 个强调色加 3 个状态色以内。
- 一个界面的字号是否不超过 4 种，标题和正文之间是否至少差一个档位。
- 是否所有文字都用语义字号，代码里有没有残留 `.system(size:)` 写死的磅值。
- 图片是否按规范裁剪：列表缩略图 56 x 56 圆角 10 裁剪填充，网格封面正方形圆角 16。
- 阴影是否基本没用，确需浮起感时是否是极轻阴影而不是默认的大黑影。
- 层级是否靠背景色差、圆角和间距表达清楚，有没有需要靠猜才能分辨的层次。

7.4 交互与反馈

- 按钮文案是否说清了点击后会怎样，有没有"确定""提交"这类含糊字样。
- 点击后是否有即时反馈：按钮状态变化、进度提示或结果提示，至少要有一样。
- 删除、清空这类破坏性操作是否二次确认，或者提供了撤销入口。
- 左滑删除后列表是否立即更新，数量提示是否同步变化。
- 保存成功后是否有确认反馈，用户能否确定数据已经存下来。
- 保存过程中是否禁止重复提交，连点两次"保存"会不会产生两条数据。
- 输入框获得焦点时键盘是否遮挡正在编辑的字段，能否自动滚动到可见区域。
- 输入为空或格式不对时，是否在对应字段旁给出提示，而不是只在控制台打印。
- 手势与滚动是否冲突，横向滑动手势有没有让列表滚不动。
- 从详情页返回后，列表的滚动位置和筛选条件是否保持原样。
- 破坏性操作的按钮颜色是否与普通按钮区分开，用户能否一眼看出风险。

7.5 适配与可访问性

- 打开深色模式后逐屏检查，有没有看不清的文字、消失的分隔线或纯白块。
- 把系统文字调到最大档，所有页面是否出现文字截断或元素重叠。
- 动态字体调大后，固定高度的容器（例如 56 点的缩略图行）是否把文字压扁。
- 在小屏幕机型上运行一遍，底部按钮有没有被安全区域或标签栏挡住。
- 安全区域是否处理正确，内容有没有跑到灵动岛、状态栏或底部指示条下面。
- 横屏下界面是否可用；不支持横屏时是否在工程设置里明确关闭，而不是让它错位。
- 横屏下键盘弹出时，输入页的可视区域是否还够用。
- VoiceOver 打开后，缩略图、角标这类纯装饰元素是否被忽略，没有读出无意义的符号名。
- VoiceOver 下每个可点元素是否都有可朗读的标签，有没有读成"按钮 按钮"。
- 用 VoiceOver 从顶部滑到底部，朗读顺序是否与视觉顺序一致，有没有跳来跳去。
- 临期、过期这类状态是否通过文字读出，而不是只靠颜色表达。
- 浅色与深色、最小与最大字号这四种组合下，是否都走完过一次完整主流程。

第 8 章 独立设计挑战

这一章不给代码。七个挑战只给需求、限制和验收标准，你要自己画线框、自己选组件、自己写代码、自己截图存档。做完这七个挑战，物品管理 App 的进阶功能（统计、筛选、分享、引导、主题）就都有原型了。

8.1 挑战规则

- 这一章不含任何实现代码，文中最多出现 API 名称的提示，具体怎么写由你决定。
- 每个挑战交三样东西：一张线框（手绘拍照或字符画 ASCII 线框，标出区块位置与大致尺寸）、一份能在模拟器跑起来的代码、一张截图（浅色深色各一张，命名成"挑战编号-界面名-模式"）。
- 线框必须在写代码之前画。先画线框能省掉大量来回改布局的时间。
- "必须包含"是功能底线，少一条算没做完；"限制条件"是逼你在约束里做设计，不是可选项。
- 七个挑战有梯度：8.2 和 8.3 练布局，8.4 和 8.5 练信息组织，8.6 和 8.7 练视觉表达，8.8 练审美判断。建议按顺序做，每题不超过两个晚上。
- 数据沿用物品管理 App 的字段：`name`、`categoryName`、`quantity`、`expireDate`、`location`、`note`、`isFavorite`、`tags`、`createdAt`、`imageData`；数值从设计规范取，间距只用 4 的倍数。

8.2 挑战一：把列表改成网格

需求：物品主页现在是一列列表。为它增加一种网格视图，用户可以在列表和网格之间切换。

必须包含：一个放在导航栏工具栏里的切换控件；网格至少显示缩略图、名称、分类和临期角标；没有图片时用统一占位块；两种视图下点击同一条都能进详情页；切换后搜索词与筛选条件不丢失。

限制条件：只能用系统组件，不许引第三方库；网格实现不超过 60 行，超出部分抽成组件；两列与三列密度都用自适应列宽；必须同时适配深色模式；缩略图在网格里必须是正方形、圆角 16。

思考提示：网格里空间更小，名称该显示几行？临期角标放右上角还是压在图下方？列表里放得下的分类名，网格里要不要换成图标？

验收标准：一屏至少看到 4 个物品；缩略图全是正方形且不变形；无图占位块与有图尺寸一致；切换视图不闪白、不重新加载数据。

8.3 挑战二：设计临期看板

需求：做一个"临期看板"，把物品按到期时间组织起来，让用户一眼看出哪些快到期、哪些已过期。

必须包含：已过期、30 天内到期、还很充裕三个分组；每组有标题和该组数量；组内按到期时间从近到远排序；每条显示名称、剩余天数和状态角标；已过期那组默认在最上方。

限制条件：分组标题必须吸顶；只能用系统组件；界面结构不超过 80 行；状态色只用规范里的绿橙红三种且只表达状态；日期不显示到秒。

思考提示：剩余天数写成"还剩 5 天"还是直接写日期？已过期的要不要显示负数天数？没有日期的物品放哪里，还是干脆不显示？

验收标准：三条不同状态的数据都落到正确分组；过期物品不算进"30 天内"；组内数量与实际条数一致；清空数据后显示空状态。

8.4 挑战三：设计筛选面板

需求：设计一个筛选面板，让用户按分类、到期状态、是否收藏、存放位置缩小列表范围。

必须包含：至少四个筛选维度，每个都能多选；顶部显示已选条件数量；提供"重置"；实时显示筛选后还剩多少条；取消后不改变原结果。

限制条件：选项必须由当前数据算出来，分类名不许写死；某维度只有两个以上选项时才显示；每个选项高度不小于 44；面板不超过 100 行；筛选逻辑放在视图之外的类型里。

思考提示：多选之间是"或"还是"与"？结果变少时，怎么让用户知道是哪个条件生效了？筛选和搜索同时存在时是叠加还是互相覆盖？

验收标准：数据里有哪几个分类，面板里就只出现哪几个；选中两个分类后结果是并集；重置后数量恢复；结果为 0 条时显示空状态并提示可重置。

8.5 挑战四：设计统计卡片

需求：给物品管理 App 做一个统计区块，用几张卡片回答"我一共有多少东西、都放在哪、有多少要过期了"。

必须包含：一张总数卡片；一张用条形或环形比例图表示的分类分布卡片；一张显示 30 天内到期数量的临期汇总卡片；每张卡片有标题、主数值和一句副标题；没有数据时都有空状态文案。

限制条件：比例图必须用 Charts 框架，不许手画；数值与副标题最多两级字号；单张卡片不超过 40 行；必须适配深色模式；数值为 0 时不能显示成空白卡片。

思考提示：主数值用大号数字还是紧凑数字？分类分布取前几名，剩下的怎么处理？几张卡片是横滑还是纵向堆叠，小屏幕上哪个更合适？

验收标准：总数与列表实际条数一致；比例图各段之和等于总数；30 天内到期数与临期看板一致；清空数据后显示 0 或空状态而不是报错。

8.6 挑战五：设计分享卡片

需求：为单个物品设计一张可以分享出去的卡片，导出成图片后发给别人也能看懂。

必须包含：物品名称、分类、数量、到期日期、一张缩略图；没有图片时用统一占位块；App 名称字样；一个二维码或一行简短说明；分享时调用系统分享面板。

限制条件：导出图片边长不小于 600 点，保证转发后文字可读；卡片必须用 App 内同一套组件渲染；分享入口放在详情页，点击区不小于 44 x 44；卡片不超过 80 行；导出时要有进行中的提示。

思考提示：卡片是正方形还是竖版长图？缩略图放顶部还是缩略成小图放左侧？被分享者看不到交互，哪些信息必须印在图上？

验收标准：导出的图片放大后文字清晰不糊；没有图片的物品导出后不留空洞；分享面板能正常弹出并保存到相册；深色模式下配色仍可读。

8.7 挑战六：设计首次使用引导

需求：为第一次打开 App 的用户设计一套引导，让他明白这个 App 能做什么、第一步该做什么。

必须包含：不超过三页，每页一句主标题加一句说明；一个随时可跳过的入口；最后一页的开始按钮直接进入添加物品；引导只在首次启动出现。

限制条件：插画只能用系统符号或简单图形；文案每句不超过 20 个汉字；引导页要有页码指示；整体不超过 100 行；结束后添加页自动弹出。

思考提示：三页分别讲清哪三件事？跳过的人是嫌长还是不感兴趣，能不能给一条更短的路径？引导里要不要放真实界面截图？

验收标准：第一次启动出现引导，第二次不再出现；跳过和走完两条路径都到主页；开始按钮直接打开添加页；文案在最小屏幕和最大字号下不截断。

8.8 挑战七：给同一个界面做两套风格

需求：挑一个你已经做好的界面（建议用物品列表页或统计页），做两套视觉风格：一套"极简"，一套"信息密集"。

必须包含：两套共用同一份数据和同一套组件；极简版强调留白，只留最关键的信息和操作；信息密集版一屏展示更多字段和入口；每套配一段不超过 100 字的说明，讲清适合什么用户和场景。

限制条件：两套都必须满足规范的硬规则（间距是 4 的倍数、可点区域 44、只用语义色和语义字号）并适配深色模式；不许靠降低可读性换取"密集"，最小字号不小于 `.caption`；两套放在同一文件的不同视图里，靠参数切换。

思考提示：极简版砍到只剩什么？密集版多出来的信息是真有用还是只是看起来专业？给长辈用和给自己用，哪一套更合适？

验收标准：两套在同一份数据上都能渲染，切换只改一个参数；密集版一屏条目不少于极简版的两倍；两套在深色模式下都通过第 7 章清单；说明能各说出一类用户和场景。

8.9 挑战的验收标准

每个挑战按四个维度评分，每项满分 5 分，总分 20 分。及格线 12 分，且布局合理性、状态完整性必须各自达到 3 分。

挑战	布局合理性	状态完整性	一致性	创意
一 列表改网格	列宽与缩略图不变形	无图、长名称、空数据都处理	与列表共用同一张卡片	切换控件的选择与动画
二 临期看板	分组层级清晰、吸顶稳定	三组全空与单组为空都有文案	状态色沿用规范三色	剩余天数的表达方式
三 筛选面板	选项可点区与选中态清楚	结果为 0、取消、重置都可预期	筛选逻辑独立于视图	维度间的与或关系
四 统计卡片	卡片内间距与对齐一致	数值为 0 时不空白	数值与图表口径统一	比例图的取用与呈现
五 分享卡片	导出后版面不散	无图与超长名称都不破版	复用 App 内组件	卡片版式与信息取舍
六 首次引导	三页节奏与页码指示	跳过、走完、二次启动都对	字号与配色沿用规范	三页各讲哪件事
七 两套风格	两套都满足硬规则	深色模式与极端字号都不破	共用同一套组件与参数	极简与密集的取舍说明

评分时先看提交物是否齐全：线框、代码、两张截图，缺一样该项按未完成计。截图要能证明你做过了深色模式。

第 9 章 设计工作表与资源

这一章是前八章的落地工具。工作表用来在写代码前把界面想清楚，速查卡用来随手核对数值，组件登记表用来防止同一个东西写三遍，资源清单用来知道去哪查资料。做新界面前先填工作表，提交前对着速查卡走一遍。

9.1 界面设计工作表模板

每设计一个新界面，先把这张表填完再打开 Xcode。示例答案来自物品列表页，照着填就知道每一行该写什么。

项目	填写内容	示例答案（物品列表页）
界面名称	与文件同名最好	物品列表页 ItemListScreen
用户最想做的事	动词开头，只写一件	快速找到某件物品，或看哪些快过期
必须出现的信息	只留做决定需要的字段	缩略图、名称、分类、临期状态、数量
可以砍掉的信息	写清这一版不做什么	备注、存放位置、标签、创建时间
布局草图	手绘或 ASCII 线框，标出区块顺序	顶部搜索栏，一行一个卡片，右下角悬浮加号
用到的组件	复用组件与新建组件	复用 ItemCard、ExpiryBadge；新建搜索栏
空状态	没数据时显示什么、给什么操作	提示"还没有物品"加"添加第一个物品"按钮
加载状态	数据没到之前显示什么	列表区域显示进度指示，不留空白
错误状态	出错时显示什么、怎么恢复	提示"读取失败"加"重试"按钮
数据量大的处理	用什么容器、要不要分页	LazyVStack 懒加载，搜索框实时过滤
验收标准	三条以内，可勾选	1. 100 条滚动不卡 2. 空数据有提示 3. 点击任意行进详情
自己的备注	留给下一版的念头	下一版加左滑收藏；默认按到期时间排序

> 提示：填不出"可以砍掉的信息"，通常意味着你正在给界面塞功能，而不是在设计界面。

9.2 设计规范速查卡

这张卡是设计规范的压缩版，取值与规范完全一致，写代码时对着它核对就够。

类别	取值	用在哪
间距	4 的倍数	所有 padding 与 spacing
图标与文字	8	图标和它的文字标签之间
页面边距	16	每个页面的左右边距
列表行留白	10	列表行的上下内边距
卡片内留白	12 或 16	卡片内部，同一屏只用一种
卡片间距	12	卡片与卡片之间
区块间距	24	分组与分组之间
圆角	10	列表缩略图、按钮、输入框
圆角	16	卡片、大图、网格封面
字号	.largeTitle	页面主标题
字号	.title3 或 .headline	区块标题
字号	.body	正文与列表主文本
字号	.subheadline 或 .footnote	次要信息
字号	.caption 或 .caption2	时间、标签、角标
文字颜色	.primary	主文字
文字颜色	.secondary	次要文字
文字颜色	.tertiary	更弱的说明
底色	Color(.systemGroupedBackground)	页面底色
底色	Color(.secondarySystemGroupedBackground)	卡片底色
强调色	根视图一句 .tint(.blue)	全局强调色，子视图不要再写
状态色	.green	正常
状态色	.orange	临期，30 天内
状态色	.red	已过期
状态色	.secondary	已归档
缩略图	56 x 56，圆角 10	列表里的图片
网格封面	正方形，圆角 16	网格卡片里的图片
详情头图	宽撑满，高 220	详情页顶部大图
可点区域	最小 44 x 44	图标按钮要加最小尺寸或 contentShape
阴影	尽量不用	需要时用黑色 6% 透明度、半径 8、下移 2

9.3 组件登记表

组件在三个以上地方用，就登记到这张表里。它回答两个问题：改这个组件会影响哪些界面，以及它提供哪些参数。

组件名	用在哪几个界面	参数	备注
ItemCard	物品列表页、搜索结果页、收藏页、网格视图	name、categoryName、image、trailing	尾部内容用 @ViewBuilder 注入；圆角固定 16
ExpiryBadge	ItemCard、物品详情页、临期看板	date	内部判断已过期与 30 天内，颜色只用红与橙，不收颜色参数
SectionHeader	临期看板、统计页、设置页	title、count、trailing	数量为零时隐藏数字；吸顶由外部容器负责

9.4 常用资源与工具

- SF Symbols App：macOS 应用商店可下载，查系统符号的准确名字和粗细变体；需要一个图标但不确定系统里有没有时用它。
- Apple 人机界面指南（Human Interface Guidelines）：域名 developer.apple.com/design，用来确认某个交互在 iOS 上的惯例做法；想引入系统里没有先例的自定义手势时先来查。
- SwiftUI 官方文档：域名 developer.apple.com/documentation/swiftui，用来确认修饰符的参数和可用版本；新 API 报版本错误时核对最低系统版本。
- WWDC 设计相关场次：developer.apple.com/videos 按年份与设计分类筛选，用来理解新控件为什么这样设计；想知道新版系统改了什么视觉规则时看。
- 配色参考工具：Coolors（coolors.co）与 Apple 系统色板文档，用来给状态色和图表配色找一组不刺眼的颜色；图表需要 4 到 6 种颜色时用它。
- 图标资源：首选 SF Symbols，缺图标时用 Lucide（lucide.dev）和 Google Material Symbols（fonts.google.com/icons）；系统符号里找不到语义贴切的图形时用它。
- 截图工具：模拟器自带的截图（Command 加 S）与 macOS 截图（Command 加 Shift 加 4），用来给挑战交付物留下浅色与深色两张图。
- 真机录屏：iOS 控制中心的屏幕录制，用来交一段 30 秒操作演示；需要证明完整流程能跑通时用它。